frontend/src/App.tsx:1-50

```
src/App.tsx:0001 import { BrowserRouter, Routes, Route, Link, useLocation, Navigate } from 'react-router-dom';
src/App.tsx:0002 import { useState, useEffect, ReactNode } from 'react';
src/App.tsx:0003 import {
src/App.tsx:0004     LayoutDashboard,
src/App.tsx:0005     Activity,
src/App.tsx:0006     Settings,
src/App.tsx:0007     Key,
src/App.tsx:0008     Server,
src/App.tsx:0009     Users,
src/App.tsx:0010     Bell,
src/App.tsx:0011     FileText,
src/App.tsx:0012     Rocket,
src/App.tsx:0013     Sun,
src/App.tsx:0014     Moon,
src/App.tsx:0015     Laptop,
src/App.tsx:0016     Radio,
src/App.tsx:0017     Brain,
src/App.tsx:0018     Network,
src/App.tsx:0019     Shield,
src/App.tsx:0020     Zap,
src/App.tsx:0021     CheckCircle,
src/App.tsx:0022     XCircle,
src/App.tsx:0023     UserCog
src/App.tsx:0024 } from 'lucide-react';
src/App.tsx:0025 import { motion, AnimatePresence } from 'framer-motion';
src/App.tsx:0026
src/App.tsx:0027 // Context
src/App.tsx:0028 import { LanguageProvider, useLanguage } from './context/LanguageContext';
src/App.tsx:0029 import { DemoProvider, useDemo } from './context/DemoContext';
src/App.tsx:0030 import { ThemeProvider, useTheme } from './context/ThemeContext';
src/App.tsx:0031 import { WebSocketProvider } from './context/WebSocketContext';
src/App.tsx:0032 import { AttackStatusProvider, useAttackStatus } from './context/AttackStatusContext';
src/App.tsx:0033 import { LiveModeProvider, useLiveMode } from './context/LiveModeContext';
src/App.tsx:0034 import Logo from './components/Logo';
src/App.tsx:0035 import type { PlanTier } from './components/Logo';
src/App.tsx:0036 import { COMPANY_NAME_CN } from './config/branding';
src/App.tsx:0037 import {
src/App.tsx:0038     fetchApiReadiness,
src/App.tsx:0039     fetchCurrentUserSession,
src/App.tsx:0040     fetchSystemBadges,
src/App.tsx:0041     isRequestAbortError
src/App.tsx:0042 } from './services/api';
src/App.tsx:0043
src/App.tsx:0044 // Pages
src/App.tsx:0045 import Dashboard from './pages/Dashboard';
src/App.tsx:0046 import Sites from './pages/Sites';
src/App.tsx:0047 import DeploymentWizard from './pages/DeploymentWizard';
src/App.tsx:0048 import Scanner from './pages/Scanner';
src/App.tsx:0049 import FixHistory from './pages/FixHistory';
src/App.tsx:0050 import SettingsPage from './pages/Settings';
```

frontend/src/App.tsx:51-100

```
src/App.tsx:0051                        import SuperAdmin from './pages/SuperAdmin';
src/App.tsx:0052                        import AdminUsers from './pages/AdminUsers';
src/App.tsx:0053                        import Alerts from './pages/Alerts';
src/App.tsx:0054                        import Rules from './pages/Rules';
src/App.tsx:0055                        import Agents from './pages/Agents';
src/App.tsx:0056                        import Devices from './pages/Devices';
src/App.tsx:0057                        import AlertDetail from './pages/AlertDetail';
src/App.tsx:0058                        import AgentDetail from './pages/AgentDetail';
src/App.tsx:0059                        import AttackRunDetail from './pages/AttackRunDetail';
src/App.tsx:0060                        import Landing from './pages/Landing';
src/App.tsx:0061                        import Login from './pages/Login';
src/App.tsx:0062                        import BoardStatus from './pages/BoardStatus';
src/App.tsx:0063                        import AgentMonitor from './pages/AgentMonitor';
src/App.tsx:0064                        import FixHistoryDetail from './pages/FixHistoryDetail';
src/App.tsx:0065                        import ScannerRunDetail from './pages/ScannerRunDetail';
src/App.tsx:0066                        import SiteDetail from './pages/SiteDetail';
src/App.tsx:0067                        import TrafficInterceptor from './pages/TrafficInterceptor';
src/App.tsx:0068                        import TrafficSessionDetail from './pages/TrafficSessionDetail';
src/App.tsx:0069                        import AttackSimulator from './pages/AttackSimulator';
src/App.tsx:0070                        import PrivacyPolicy from './pages/PrivacyPolicy';
src/App.tsx:0071                        import TermsOfService from './pages/TermsOfService';
src/App.tsx:0072                        import Promo from './pages/Promo';
src/App.tsx:0073                        import PromoFilm from './pages/PromoFilm';
src/App.tsx:0074
src/App.tsx:0075                        /**
src/App.tsx:0076                         * SECURITY FIX CVE-2026-012: Authentication guard for protected routes
src/App.tsx:0077                         * Improved cold start handling
src/App.tsx:0078                         */
src/App.tsx:0079                        const PrivateRoute = ({ children }: { children: ReactNode }) => {
src/App.tsx:0080                            const [isAuthenticated, setIsAuthenticated] = useState<boolean | null>(null);
src/App.tsx:0081                            const [backendReady, setBackendReady] = useState(false);
src/App.tsx:0082
src/App.tsx:0083                            useEffect(() => {
src/App.tsx:0084                                // Check backend readiness first (lightweight health check)
src/App.tsx:0085                                const checkBackend = async () => {
src/App.tsx:0086                                    try {
src/App.tsx:0087                                        const readiness = await fetchApiReadiness(AbortSignal.timeout(3000));
src/App.tsx:0088                                        if (readiness.ok) {
src/App.tsx:0089                                            setBackendReady(true);
src/App.tsx:0090                                            const user = await fetchCurrentUserSession(AbortSignal.timeout(5000));
src/App.tsx:0091                                            setIsAuthenticated(Boolean(user));
src/App.tsx:0092                                        } else {
src/App.tsx:0093                                            setBackendReady(true);
src/App.tsx:0094                                            setIsAuthenticated(false);
src/App.tsx:0095                                        }
src/App.tsx:0096                                    } catch {
src/App.tsx:0097                                        // Backend not ready - don't block UI, show login
src/App.tsx:0098                                        setBackendReady(true); // Allow access to check auth
src/App.tsx:0099                                        setIsAuthenticated(false);
src/App.tsx:0100                                    }
```

frontend/src/App.tsx:101-150

```
src/App.tsx:0101                                };
src/App.tsx:0102                                checkBackend();
src/App.tsx:0103                            }, []);
src/App.tsx:0104
src/App.tsx:0105                            // Show loading while checking backend and auth
src/App.tsx:0106                            if (!backendReady || isAuthenticated === null) {
src/App.tsx:0107                                return (
src/App.tsx:0108                                    <div className="flex items-center justify-center h-screen bg-background">
src/App.tsx:0109                                        <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-primary"></div>
src/App.tsx:0110                                    </div>
src/App.tsx:0111                                );
src/App.tsx:0112                            }
src/App.tsx:0113
src/App.tsx:0114                            // Redirect to login if not authenticated
src/App.tsx:0115                            if (!isAuthenticated) {
src/App.tsx:0116                                return <Navigate to="/login" replace />;
src/App.tsx:0117                            }
src/App.tsx:0118
src/App.tsx:0119                            return <>{children}</>;
src/App.tsx:0120                        };
src/App.tsx:0121
src/App.tsx:0122                        /**
src/App.tsx:0123                         * SECURITY FIX CVE-2026-012: Admin-only route guard
src/App.tsx:0124                         * Improved cold start handling
src/App.tsx:0125                         */
src/App.tsx:0126                        const AdminRoute = ({ children }: { children: ReactNode }) => {
src/App.tsx:0127                            const [userRole, setUserRole] = useState<string | null>(null);
src/App.tsx:0128                            const [isLoading, setIsLoading] = useState(true);
src/App.tsx:0129
src/App.tsx:0130                            useEffect(() => {
src/App.tsx:0131                                const checkRole = async () => {
src/App.tsx:0132                                    try {
src/App.tsx:0133                                        const user = await fetchCurrentUserSession(AbortSignal.timeout(5000));
src/App.tsx:0134                                        setUserRole(user?.role || null);
src/App.tsx:0135                                    } catch {
src/App.tsx:0136                                        setUserRole(null);
src/App.tsx:0137                                    }
src/App.tsx:0138                                    setIsLoading(false);
src/App.tsx:0139                                };
src/App.tsx:0140                                checkRole();
src/App.tsx:0141                            }, []);
src/App.tsx:0142
src/App.tsx:0143                            if (isLoading) {
src/App.tsx:0144                                return (
src/App.tsx:0145                                    <div className="flex items-center justify-center h-screen bg-background">
src/App.tsx:0146                                        <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-primary"></div>
src/App.tsx:0147                                    </div>
src/App.tsx:0148                                );
src/App.tsx:0149                            }
src/App.tsx:0150
```

frontend/src/App.tsx:151-200

```
src/App.tsx:0151                            if (userRole !== 'admin') {
src/App.tsx:0152                                return <Navigate to="/dashboard" replace />;
src/App.tsx:0153                            }
src/App.tsx:0154
src/App.tsx:0155                            return <>{children}</>;
src/App.tsx:0156                        };
src/App.tsx:0157
src/App.tsx:0158                        const Sidebar = () => {
src/App.tsx:0159                            const location = useLocation();
src/App.tsx:0160                            const { t } = useLanguage();
src/App.tsx:0161                            const { status: attackStatus } = useAttackStatus();
src/App.tsx:0162                            const { attackSimEnabled } = useDemo();
src/App.tsx:0163                            const { liveModeEnabled } = useLiveMode();
src/App.tsx:0164                            const [badges, setBadges] = useState({ agents: 0, alerts: 0, devices: 0, agentMonitor: 0, traffic: 0, scanner: 0 });
src/App.tsx:0165                            const [profile, setProfile] = useState<{ name: string; role: string; initials: string; plan: string }>({
src/App.tsx:0166                                name: 'Unknown User',
src/App.tsx:0167                                role: 'N/A',
src/App.tsx:0168                                initials: '??',
src/App.tsx:0169                                plan: 'starter'
src/App.tsx:0170                            });
src/App.tsx:0171
src/App.tsx:0172                            // Don't show sidebar on standalone public pages
src/App.tsx:0173                            if (location.pathname === '/' || location.pathname === '/login' || location.pathname === '/privacy' || location.pathname === '/terms' || location
src/App.tsx:0174
src/App.tsx:0175                            const platformItems = [
src/App.tsx:0176                                { icon: LayoutDashboard, label: t('nav.dashboard'), path: '/dashboard' },
src/App.tsx:0177                                { icon: Users, label: t('nav.agents'), path: '/agents', badge: badges.agents > 0 ? badges.agents : undefined },
src/App.tsx:0178                                { icon: Brain, label: t('nav.agent_monitor'), path: '/agent-monitor', badge: badges.agentMonitor > 0 ? badges.agentMonitor : undefined },
src/App.tsx:0179                                { icon: Network, label: t('nav.traffic_interceptor'), path: '/traffic', badge: badges.traffic > 0 ? badges.traffic : undefined },
src/App.tsx:0180                                ...(attackSimEnabled ? [{ icon: Shield, label: t('nav.attack_simulator'), path: '/attack-sim' }] : []),
src/App.tsx:0181                                { icon: Radio, label: t('nav.devices'), path: '/devices', badge: badges.devices > 0 ? badges.devices : undefined },
src/App.tsx:0182                                { icon: Server, label: t('nav.sites'), path: '/sites' },
src/App.tsx:0183                                { icon: Rocket, label: t('nav.deployment'), path: '/deploy' },
src/App.tsx:0184                                { icon: Activity, label: t('nav.monitoring'), path: '/scanner', badge: badges.scanner > 0 ? badges.scanner : undefined },
src/App.tsx:0185                                { icon: Settings, label: t('nav.settings'), path: '/settings' },
src/App.tsx:0186                                { icon: Key, label: t('nav.keys'), path: '/admin' },
src/App.tsx:0187                                ...(profile.role === 'admin' ? [{ icon: UserCog, label: t('nav.super_admin'), path: '/super-admin' }] : []),
src/App.tsx:0188                            ];
src/App.tsx:0189
src/App.tsx:0190                            const navGroups = [
src/App.tsx:0191                                {
src/App.tsx:0192                                    title: t('nav.platform'),
src/App.tsx:0193                                    items: platformItems
src/App.tsx:0194                                },
src/App.tsx:0195                                {
src/App.tsx:0196                                    title: t('nav.governance'),
src/App.tsx:0197                                    items: [
src/App.tsx:0198                                        { icon: FileText, label: t('nav.audit'), path: '/history' },
src/App.tsx:0199                                        ...(profile.role === 'admin' ? [{ icon: Shield, label: 'Rules', path: '/rules' }] : []),
src/App.tsx:0200                                        { icon: Bell, label: t('nav.alerts'), path: '/alerts', badge: badges.alerts > 0 ? badges.alerts : undefined },
```

frontend/src/App.tsx:201-250

```
src/App.tsx:0201                                ]
src/App.tsx:0202                            }
src/App.tsx:0203                        ];
src/App.tsx:0204
src/App.tsx:0205                        const progressPercent = attackStatus.total > 0
src/App.tsx:0206                            ? Math.round((attackStatus.completed / attackStatus.total) * 100)
src/App.tsx:0207                            : 0;
src/App.tsx:0208                        const logoTier: PlanTier =
src/App.tsx:0209                            profile.plan === 'pro'
src/App.tsx:0210                                ? 'professional'
src/App.tsx:0211                                : profile.plan === 'ultra'
src/App.tsx:0212                                    ? 'enterprise'
src/App.tsx:0213                                    : 'starter';
src/App.tsx:0214                        type NavItem = {
src/App.tsx:0215                            icon: typeof LayoutDashboard;
src/App.tsx:0216                            label: string;
src/App.tsx:0217                            path: string;
src/App.tsx:0218                            badge?: number;
src/App.tsx:0219                        };
src/App.tsx:0220
src/App.tsx:0221                        useEffect(() => {
src/App.tsx:0222                            let disposed = false;
src/App.tsx:0223                            let activeController: AbortController | null = null;
src/App.tsx:0224
src/App.tsx:0225                            const loadBadges = async () => {
src/App.tsx:0226                                const controller = new AbortController();
src/App.tsx:0227                                activeController?.abort();
src/App.tsx:0228                                activeController = controller;
src/App.tsx:0229
src/App.tsx:0230                                try {
src/App.tsx:0231                                    const data = await fetchSystemBadges(controller.signal);
src/App.tsx:0232                                    if (disposed || controller.signal.aborted) {
src/App.tsx:0233                                        return;
src/App.tsx:0234                                    }
src/App.tsx:0235                                    setBadges({
src/App.tsx:0236                                        agents: data.agents || 0,
src/App.tsx:0237                                        alerts: data.alerts || 0,
src/App.tsx:0238                                        devices: data.devices || 0,
src/App.tsx:0239                                        agentMonitor: data.agentMonitor || 0,
src/App.tsx:0240                                        traffic: data.traffic || 0,
src/App.tsx:0241                                        scanner: data.scanner || 0,
src/App.tsx:0242                                    });
src/App.tsx:0243                                } catch (e) {
src/App.tsx:0244                                    if (!isRequestAbortError(e, controller.signal)) {
src/App.tsx:0245                                        // ignore badge polling failures here; page-level fallbacks already handle empty badges
src/App.tsx:0246                                    }
src/App.tsx:0247                                }
src/App.tsx:0248                            };
src/App.tsx:0249
src/App.tsx:0250                            void loadBadges();
```

frontend/src/App.tsx:251-300

```
src/App.tsx:0251                                if (!liveModeEnabled) {
src/App.tsx:0252                                    return () => {
src/App.tsx:0253                                        disposed = true;
src/App.tsx:0254                                        activeController?.abort();
src/App.tsx:0255                                    };
src/App.tsx:0256                                }
src/App.tsx:0257
src/App.tsx:0258                                const badgeInterval = setInterval(() => {
src/App.tsx:0259                                    void loadBadges();
src/App.tsx:0260                                }, 4000);
src/App.tsx:0261
src/App.tsx:0262                                return () => {
src/App.tsx:0263                                    disposed = true;
src/App.tsx:0264                                    activeController?.abort();
src/App.tsx:0265                                    clearInterval(badgeInterval);
src/App.tsx:0266                                };
src/App.tsx:0267                            }, [liveModeEnabled]);
src/App.tsx:0268
src/App.tsx:0269                            useEffect(() => {
src/App.tsx:0270                                const controller = new AbortController();
src/App.tsx:0271
src/App.tsx:0272                                fetchCurrentUserSession(controller.signal)
src/App.tsx:0273                                    .then((user) => {
src/App.tsx:0274                                        if (controller.signal.aborted) {
src/App.tsx:0275                                            return;
src/App.tsx:0276                                        }
src/App.tsx:0277                                        if (!user) {
src/App.tsx:0278                                            throw new Error('FG-AUTH-ME-401');
src/App.tsx:0279                                        }
src/App.tsx:0280                                        const name = user.username || user.email || '未命名用户';
src/App.tsx:0281                                        const initials = String(name).split(/\s+/).map((x: string) => x[0] || '').join('').slice(0, 2).toUpperCase() || '??';
src/App.tsx:0282                                        setProfile({
src/App.tsx:0283                                            name,
src/App.tsx:0284                                            role: user.role || '未设置角色',
src/App.tsx:0285                                            initials,
src/App.tsx:0286                                            plan: user.plan || 'starter'
src/App.tsx:0287                                        });
src/App.tsx:0288                                    })
src/App.tsx:0289                                    .catch((error) => {
src/App.tsx:0290                                        if (isRequestAbortError(error, controller.signal)) {
src/App.tsx:0291                                            return;
src/App.tsx:0292                                        }
src/App.tsx:0293                                        setProfile({ name: '未命名用户', role: '未设置角色', initials: '??', plan: 'starter' });
src/App.tsx:0294                                    });
src/App.tsx:0295
src/App.tsx:0296                                return () => controller.abort();
src/App.tsx:0297                            }, []);
src/App.tsx:0298
src/App.tsx:0299                            return (
src/App.tsx:0300                                <aside className="z-20 hidden w-64 flex-col justify-between border-r border-border bg-card md:flex">
```

frontend/src/App.tsx:301-350

```
src/App.tsx:0301                                    <div>
src/App.tsx:0302                                        {/* Logo */}
src/App.tsx:0303                                        <div className="border-b border-border/50 px-6 py-3">
src/App.tsx:0304                                            <div className="space-y-1">
src/App.tsx:0305                                                <Logo
src/App.tsx:0306                                                    tier={logoTier}
src/App.tsx:0307                                                    showText={true}
src/App.tsx:0308                                                    className="origin-left scale-90"
src/App.tsx:0309                                                />
src/App.tsx:0310                                                <div className="pl-1 text-[11px] text-muted-foreground">{COMPANY NAME CN}</div>
src/App.tsx:0311                                            </div>
src/App.tsx:0312                                        </div>
src/App.tsx:0313
src/App.tsx:0314                                        {/* Nav Links */}
src/App.tsx:0315                                        <nav className="p-3 space-y-4">
src/App.tsx:0316                                            {navGroups.map((group, idx) => (
src/App.tsx:0317                                                <div key={idx}>
src/App.tsx:0318                                                    <div className="px-3 py-2 text-xxs font-medium tracking-wider text-muted-foreground uppercase">{group.title}</div>
src/App.tsx:0319                                                    <div className="space-y-0.5">
src/App.tsx:0320                                                        {group.items.map((item: NavItem) => {
src/App.tsx:0321                                                            const isActive = location.pathname === item.path;
src/App.tsx:0322                                                            return (
src/App.tsx:0323                                                                <Link
src/App.tsx:0324                                                                    key={item.path}
src/App.tsx:0325                                                                    to={item.path}
src/App.tsx:0326                                                                    className={`flex items-center gap-3 px-3 py-2 text-sm rounded-md transition-colors group ${isActive
src/App.tsx:0327                                                                        ? 'text-foreground bg-primary/10 border border-primary/20 shadow-sm'
src/App.tsx:0328                                                                        : 'text-muted-foreground hover:text-foreground hover:bg-muted'
src/App.tsx:0329                                                                        }`}
src/App.tsx:0330                                                                >
src/App.tsx:0331                                                                    <item.icon
src/App.tsx:0332                                                                        size={18}
src/App.tsx:0333                                                                        strokeWidth={1.5}
src/App.tsx:0334                                                                        className={isActive ? 'text-primary' : 'group-hover:text-foreground'}
src/App.tsx:0335                                                                    />
src/App.tsx:0336                                                                    <span className="font-normal">{item.label}</span>
src/App.tsx:0337                                                                    {item.path === '/scanner' && (
src/App.tsx:0338                                                                        <span className="ml-auto flex h-1.5 w-1.5 rounded-full bg-emerald-500"></span>
src/App.tsx:0339                                                                    )}
src/App.tsx:0340                                                                    {item.badge && (
src/App.tsx:0341                                                                        <span className="ml-auto text-xxs bg-red-500/10 text-red-400 px-1.5 py-0.5 rounded border border-red-
src/App.tsx:0342                                                                    )}
src/App.tsx:0343                                                                </Link>
src/App.tsx:0344                                                            );
src/App.tsx:0345                                                        })}
src/App.tsx:0346                                                    </div>
src/App.tsx:0347                                                </div>
src/App.tsx:0348                                            ))}
src/App.tsx:0349                                        </nav>
src/App.tsx:0350                                    </div>
```

frontend/src/App.tsx:351-400

```
src/App.tsx:0351
src/App.tsx:0352                                    <div className="p-4 border-t border-border/50 space-y-3">
src/App.tsx:0353                                        {/* Attack Status */}
src/App.tsx:0354                                        <AnimatePresence>
src/App.tsx:0355                                            {attackSimEnabled && attackStatus.isRunning && (
src/App.tsx:0356                                                <motion.div
src/App.tsx:0357                                                    initial={{ opacity: 0, height: 0 }}
src/App.tsx:0358                                                    animate={{ opacity: 1, height: 'auto' }}
src/App.tsx:0359                                                    exit={{ opacity: 0, height: 0 }}
src/App.tsx:0360                                                    className="overflow-hidden"
src/App.tsx:0361                                                >
src/App.tsx:0362                                                    <div className="rounded-lg border border-border bg-muted/20 p-3">
src/App.tsx:0363                                                        <div className="flex items-center gap-2 mb-2">
src/App.tsx:0364                                                            <Zap className="h-4 w-4 text-amber-500" />
src/App.tsx:0365                                                            <span className="text-xs font-medium text-foreground">{t('nav.attack_simulation')}</span>
src/App.tsx:0366                                                        </div>
src/App.tsx:0367
src/App.tsx:0368                                                        <div className="space-y-2">
src/App.tsx:0369                                                            <div className="flex justify-between text-xs">
src/App.tsx:0370                                                                <span className="text-muted-foreground">{t('nav.attack sim progress')}</span>
src/App.tsx:0371                                                                <span className="font-medium text-foreground">{attackStatus.completed}/{attackStatus.total}</span>
src/App.tsx:0372                                                            </div>
src/App.tsx:0373                                                            <div className="h-1.5 overflow-hidden rounded-full bg-muted">
src/App.tsx:0374                                                                <motion.div
src/App.tsx:0375                                                                    initial={{ width: 0 }}
src/App.tsx:0376                                                                    animate={{ width: `${progressPercent}%` }}
src/App.tsx:0377                                                                    className="h-full rounded-full bg-amber-500"
src/App.tsx:0378                                                                />
src/App.tsx:0379                                                            </div>
src/App.tsx:0380
src/App.tsx:0381                                                            <div className="flex items-center justify-between pt-1">
src/App.tsx:0382                                                                <div className="flex items-center gap-1">
src/App.tsx:0383                                                                    <CheckCircle className="h-3 w-3 text-emerald-500" />
src/App.tsx:0384                                                                    <span className="text-xs text-emerald-600 dark:text-emerald-400">{attackStatus.blocked}</span>
src/App.tsx:0385                                                                </div>
src/App.tsx:0386                                                                <div className="flex items-center gap-1">
src/App.tsx:0387                                                                    <XCircle className="h-3 w-3 text-rose-500" />
src/App.tsx:0388                                                                    <span className="text-xs text-rose-600 dark:text-rose-400">{attackStatus.allowed}</span>
src/App.tsx:0389                                                                </div>
src/App.tsx:0390                                                            </div>
src/App.tsx:0391
src/App.tsx:0392                                                            {attackStatus.currentAttack && (
src/App.tsx:0393                                                                <div className="truncate border-t border-border/60 pt-1 text-xs text-muted-foreground">
src/App.tsx:0394                                                                    {attackStatus.currentAttack}
src/App.tsx:0395                                                                </div>
src/App.tsx:0396                                                            )}
src/App.tsx:0397                                                        </div>
src/App.tsx:0398                                                    </div>
src/App.tsx:0399                                                </motion.div>
src/App.tsx:0400                                            )}
```

frontend/src/App.tsx:401-450

```
src/App.tsx:0401                                    </AnimatePresence>
src/App.tsx:0402
src/App.tsx:0403                                    {/* User Profile */}
src/App.tsx:0404                                    <Link to="/settings" className="flex items-center gap-3 px-2 py-2 text-sm hover:text-foreground transition-colors">
src/App.tsx:0405                                        <div className="w-8 h-8 rounded-full bg-primary/20 border border-primary/30 flex items-center justify-center text-primary font-me
src/App.tsx:0406                                            {profile.initials}
src/App.tsx:0407                                        </div>
src/App.tsx:0408                                        <div className="flex flex-col">
src/App.tsx:0409                                            <span className="text-xs font-medium text-foreground">{profile.name}</span>
src/App.tsx:0410                                            <span className="text-xxs text-muted-foreground">{profile.role}</span>
src/App.tsx:0411                                        </div>
src/App.tsx:0412                                        <Settings size={18} className="ml-auto hover:rotate-90 transition-transform duration-500 text-muted-foreground hover:text-foregro
src/App.tsx:0413                                    </Link>
src/App.tsx:0414                                </div>
src/App.tsx:0415                            </aside>
src/App.tsx:0416                        );
src/App.tsx:0417                    }
src/App.tsx:0418
src/App.tsx:0419                    const Header = () => {
src/App.tsx:0420                        const { t, language, toggleLanguage } = useLanguage();
src/App.tsx:0421                        const { isDemoMode, toggleDemoMode, isSwitchingDemoMode } = useDemo();
src/App.tsx:0422                        const { theme, setTheme } = useTheme();
src/App.tsx:0423                        const { liveModeEnabled, toggleLiveMode } = useLiveMode();
src/App.tsx:0424                        const location = useLocation();
src/App.tsx:0425
src/App.tsx:0426                        // Don't show header on standalone public pages
src/App.tsx:0427                        if (location.pathname === '/' || location.pathname === '/login' || location.pathname === '/privacy' || location.pathname === '/terms' || location
src/App.tsx:0428
src/App.tsx:0429                        return (
src/App.tsx:0430                            <header className="sticky top-0 z-10 flex h-16 items-center justify-between gap-3 border-b border-border/50 bg-background px-3 md:px-6">
src/App.tsx:0431                                <div className="min-w-0 flex items-center gap-3 text-sm text-muted-foreground overflow-hidden">
src/App.tsx:0432                                    <span className="hover:text-foreground cursor-pointer transition-colors truncate">{t('header.overview')}</span>
src/App.tsx:0433                                    <span className="text-muted-foreground/50 shrink-0">/</span>
src/App.tsx:0434                                    <span className="text-foreground font-medium truncate">{t('nav.dashboard')}</span>
src/App.tsx:0435                                </div>
src/App.tsx:0436
src/App.tsx:0437                                <div className="flex items-center gap-2 md:gap-4 shrink-0">
src/App.tsx:0438                                    {/* Status Indicator */}
src/App.tsx:0439                                    <div className="flex items-center gap-2 rounded-full border border-emerald-500/10 bg-emerald-500/5 px-3 py-1.5">
src/App.tsx:0440                                        <span className="inline-flex h-2 w-2 rounded-full bg-emerald-500"></span>
src/App.tsx:0441                                        <span className="hidden lg:inline text-xs font-medium text-emerald-500 tracking-tight">{t('header.system_healthy')}</span>
src/App.tsx:0442                                    </div>
src/App.tsx:0443
src/App.tsx:0444                                    <div className="hidden sm:block h-8 w-px bg-border"></div>
src/App.tsx:0445
src/App.tsx:0446                                    {/* Theme Toggle */}
src/App.tsx:0447                                    <div className="flex items-center gap-1 bg-muted border border-border rounded-full p-0.5">
src/App.tsx:0448                                        <button
src/App.tsx:0449                                            onClick={() => setTheme('light')}
src/App.tsx:0450                                            className={`p-1 rounded-full transition-all ${theme === 'light' ? 'bg-background shadow text-primary' : 'text-muted-foregroun
```

frontend/src/App.tsx:451-500

```
src/App.tsx:0451                                        title="Light Mode"
src/App.tsx:0452                                    >
src/App.tsx:0453                                        <Sun size={14} />
src/App.tsx:0454                                    </button>
src/App.tsx:0455                                    <button
src/App.tsx:0456                                        onClick={() => setTheme('system')}
src/App.tsx:0457                                        className={`p-1 rounded-full transition-all ${theme === 'system' ? 'bg-background text-foreground shadow' : 'text-muted-foreg
src/App.tsx:0458                                        title="System"
src/App.tsx:0459                                    >
src/App.tsx:0460                                        <Laptop size={14} />
src/App.tsx:0461                                    </button>
src/App.tsx:0462                                    <button
src/App.tsx:0463                                        onClick={() => setTheme('dark')}
src/App.tsx:0464                                        className={`p-1 rounded-full transition-all ${theme === 'dark' ? 'bg-background text-primary shadow' : 'text-muted-foreground
src/App.tsx:0465                                        title="Dark Mode"
src/App.tsx:0466                                    >
src/App.tsx:0467                                        <Moon size={14} />
src/App.tsx:0468                                    </button>
src/App.tsx:0469                                </div>
src/App.tsx:0470
src/App.tsx:0471                                <div className="hidden sm:block h-8 w-px bg-border"></div>
src/App.tsx:0472
src/App.tsx:0473                                <button
src/App.tsx:0474                                    type="button"
src/App.tsx:0475                                    data-testid="demo-mode-toggle"
src/App.tsx:0476                                    aria-pressed={isDemoMode}
src/App.tsx:0477                                    onClick={toggleDemoMode}
src/App.tsx:0478                                    disabled={isSwitchingDemoMode}
src/App.tsx:0479                                    className={`flex items-center gap-2 group shrink-0 whitespace-nowrap ${isSwitchingDemoMode ? 'opacity-60 cursor-not-allowed' : 'c
src/App.tsx:0480                                >
src/App.tsx:0481                                    <span className={`text-xs font-medium transition-colors ${isDemoMode ? 'text-primary' : 'text-muted-foreground group-hover:text-f
src/App.tsx:0482                                    <div className={`w-8 h-4 rounded-full relative transition-colors border ${isDemoMode ? 'bg-primary/20 border-primary/50' : 'bg-mu
src/App.tsx:0483                                        <div className={`absolute top-0.5 w-3 h-3 rounded-full shadow-sm transition-all ${isDemoMode ? 'right-0.5 bg-primary' : 'left
src/App.tsx:0484                                    </div>
src/App.tsx:0485                                </button>
src/App.tsx:0486
src/App.tsx:0487                                <div className="hidden sm:block h-8 w-px bg-border"></div>
src/App.tsx:0488
src/App.tsx:0489                                {/* Language Toggle */}
src/App.tsx:0490                                <div onClick={toggleLanguage} className="flex items-center gap-2 cursor-pointer group">
src/App.tsx:0491                                    <span className={`text-xs font-medium transition-colors ${language === 'zh' ? 'text-muted-foreground' : 'text-foreground'}`}>EN</
src/App.tsx:0492                                    <div className={`w-8 h-4 rounded-full relative transition-colors ${language === 'zh' ? 'bg-primary/20 border-primary/50' : 'bg-mu
src/App.tsx:0493                                        <div className={`absolute top-0.5 w-3 h-3 rounded-full shadow-sm transition-all ${language === 'zh' ? 'right-0.5 bg-primary'
src/App.tsx:0494                                    </div>
src/App.tsx:0495                                    <span className={`text-xs font-medium transition-colors ${language === 'zh' ? 'text-primary' : 'text-muted-foreground'}`}>中文</s
src/App.tsx:0496                                </div>
src/App.tsx:0497
src/App.tsx:0498                                <div className="h-8 w-px bg-border"></div>
src/App.tsx:0499
src/App.tsx:0500                                {/* Custom Switch (Auto Refresh) */}
```

frontend/src/App.tsx:501-550

```
src/App.tsx:0501                                    <button type="button" onClick={toggleLiveMode} className="flex items-center gap-2 group cursor-pointer">
src/App.tsx:0502                                        <span className="text-xs font-medium group-hover:text-foreground transition-colors text-muted-foreground">{t('header.live')}</spa
src/App.tsx:0503                                        <div className={`w-9 h-5 rounded-full border relative transition-colors ${liveModeEnabled ? 'bg-primary/20 border-primary/50' : '
src/App.tsx:0504                                            <div className={`absolute top-0.5 w-3.5 h-3.5 rounded-full shadow-sm transition-all ${liveModeEnabled ? 'right-0.5 bg-primary
src/App.tsx:0505                                        </div>
src/App.tsx:0506                                    </button>
src/App.tsx:0507                                </div>
src/App.tsx:0508                            </header>
src/App.tsx:0509                        )
src/App.tsx:0510                    }
src/App.tsx:0511
src/App.tsx:0512                    const PageWrapper = ({ children }: { children: React.ReactNode }) => (
src/App.tsx:0513                        <motion.div
src/App.tsx:0514                            initial={{ opacity: 0, scale: 0.98 }}
src/App.tsx:0515                            animate={{ opacity: 1, scale: 1 }}
src/App.tsx:0516                            exit={{ opacity: 0, scale: 0.98 }}
src/App.tsx:0517                            transition={{ duration: 0.2 }}
src/App.tsx:0518                            className="h-full"
src/App.tsx:0519                        >
src/App.tsx:0520                            {children}
src/App.tsx:0521                        </motion.div>
src/App.tsx:0522                    );
src/App.tsx:0523
src/App.tsx:0524                    const AnimatedRoutes = () => {
src/App.tsx:0525                        const location = useLocation();
src/App.tsx:0526                        const { attackSimEnabled, modeResolved } = useDemo();
src/App.tsx:0527
src/App.tsx:0528                        return (
src/App.tsx:0529                            <AnimatePresence mode="wait">
src/App.tsx:0530                                <Routes location={location} key={location.pathname}>
src/App.tsx:0531                                    {/* Public routes */}
src/App.tsx:0532                                    <Route path="/" element={<PageWrapper><Landing /></PageWrapper>} />
src/App.tsx:0533                                    <Route path="/promo" element={<Promo />} />
src/App.tsx:0534                                    <Route path="/promo-film" element={<PromoFilm />} />
src/App.tsx:0535                                    <Route path="/login" element={<PageWrapper><Login /></PageWrapper>} />
src/App.tsx:0536                                    <Route path="/privacy" element={<PageWrapper><PrivacyPolicy /></PageWrapper>} />
src/App.tsx:0537                                    <Route path="/terms" element={<PageWrapper><TermsOfService /></PageWrapper>} />
src/App.tsx:0538
src/App.tsx:0539                                    {/* SECURITY FIX CVE-2026-012: Protected routes require authentication */}
src/App.tsx:0540                                    <Route path="/dashboard" element={<PrivateRoute><PageWrapper><Dashboard /></PageWrapper></PrivateRoute>} />
src/App.tsx:0541                                    <Route path="/agents" element={<PrivateRoute><PageWrapper><Agents /></PageWrapper></PrivateRoute>} />
src/App.tsx:0542                                    <Route path="/agent-monitor" element={<PrivateRoute><PageWrapper><AgentMonitor /></PageWrapper></PrivateRoute>} />
src/App.tsx:0543                                    <Route path="/traffic" element={<PrivateRoute><PageWrapper><TrafficInterceptor /></PageWrapper></PrivateRoute>} />
src/App.tsx:0544                                    <Route
src/App.tsx:0545                                        path="/attack-sim"
src/App.tsx:0546                                        element={
src/App.tsx:0547                                            !modeResolved
src/App.tsx:0548                                                ? <PrivateRoute><PageWrapper><div className="p-8 text-muted-foreground">Loading mode...</div></PageWrapper></PrivateRoute
src/App.tsx:0549                                                : attackSimEnabled
src/App.tsx:0550                                                    ? <PrivateRoute><PageWrapper><AttackSimulator /></PageWrapper></PrivateRoute>
```

frontend/src/App.tsx:551-574 | frontend/src/pages/Dashboard.tsx:1-26

```
src/App.tsx:0551                                                    : <Navigate to="/dashboard" replace />
src/App.tsx:0552                                                }
src/App.tsx:0553                                            />
src/App.tsx:0554                                            <Route path="/devices" element={<PrivateRoute><PageWrapper><Devices /></PageWrapper></PrivateRoute>} />
src/App.tsx:0555                                            <Route path="/sites" element={<PrivateRoute><PageWrapper><Sites /></PageWrapper></PrivateRoute>} />
src/App.tsx:0556                                            <Route path="/deploy" element={<PrivateRoute><PageWrapper><DeploymentWizard /></PageWrapper></PrivateRoute>} />
src/App.tsx:0557                                            <Route path="/scanner" element={<PrivateRoute><PageWrapper><Scanner /></PageWrapper></PrivateRoute>} />
src/App.tsx:0558                                            <Route path="/scanner/runs/:id" element={<PrivateRoute><PageWrapper><ScannerRunDetail /></PageWrapper></PrivateRoute>} />
src/App.tsx:0559                                            <Route path="/history" element={<PrivateRoute><PageWrapper><FixHistory /></PageWrapper></PrivateRoute>} />
src/App.tsx:0560                                            <Route path="/history/:id" element={<PrivateRoute><PageWrapper><FixHistoryDetail /></PageWrapper></PrivateRoute>} />
src/App.tsx:0561                                            <Route path="/settings" element={<PrivateRoute><PageWrapper><SettingsPage /></PageWrapper></PrivateRoute>} />
src/App.tsx:0562                                            <Route path="/alerts" element={<PrivateRoute><PageWrapper><Alerts /></PageWrapper></PrivateRoute>} />
src/App.tsx:0563                                            <Route path="/alerts/:id" element={<PrivateRoute><PageWrapper><AlertDetail /></PageWrapper></PrivateRoute>} />
src/App.tsx:0564                                            <Route path="/rules" element={<PrivateRoute><AdminRoute><PageWrapper><Rules /></PageWrapper></AdminRoute></PrivateRoute>} />
src/App.tsx:0565                                            <Route path="/agents/:id" element={<PrivateRoute><PageWrapper><AgentDetail /></PageWrapper></PrivateRoute>} />
src/App.tsx:0566                                            <Route path="/devices/board-status/:id" element={<PrivateRoute><PageWrapper><BoardStatus /></PageWrapper></PrivateRoute>} />
src/App.tsx:0567                                            <Route path="/sites/:id" element={<PrivateRoute><PageWrapper><SiteDetail /></PageWrapper></PrivateRoute>} />
src/App.tsx:0568                                            <Route path="/traffic/sessions/:id" element={<PrivateRoute><PageWrapper><TrafficSessionDetail /></PageWrapper></PrivateRoute>} />
src/App.tsx:0569                                            <Route path="/attack-sim/runs/:id" element={<PrivateRoute><PageWrapper><AttackRunDetail /></PageWrapper></PrivateRoute>} />
src/App.tsx:0570
src/App.tsx:0571                                            {/* Admin-only routes */}
src/App.tsx:0572                                            <Route path="/admin" element={<PrivateRoute><AdminRoute><PageWrapper><SuperAdmin /></PageWrapper></AdminRoute></PrivateRoute>} />
src/App.tsx:0573                                            <Route path="/super-admin" element={<PrivateRoute><AdminRoute><PageWrapper><AdminUsers /></PageWrapper></AdminRoute></PrivateRoute>}
src/App.tsx:0574                                        </Routes>
src/pages/Dashboard.tsx:0001            import { useEffect, useMemo, useState } from 'react';
src/pages/Dashboard.tsx:0002            import { motion, AnimatePresence } from 'framer-motion';
src/pages/Dashboard.tsx:0003            import { AreaChart, Area, XAxis, YAxis, CartesianGrid, Tooltip } from 'recharts';
src/pages/Dashboard.tsx:0004            import {
src/pages/Dashboard.tsx:0005                Activity,
src/pages/Dashboard.tsx:0006                Code,
src/pages/Dashboard.tsx:0007                Cpu,
src/pages/Dashboard.tsx:0008                Database,
src/pages/Dashboard.tsx:0009                Download,
src/pages/Dashboard.tsx:0010                Globe,
src/pages/Dashboard.tsx:0011                HardDrive,
src/pages/Dashboard.tsx:0012                Key,
src/pages/Dashboard.tsx:0013                Pause,
src/pages/Dashboard.tsx:0014                Search,
src/pages/Dashboard.tsx:0015                Server,
src/pages/Dashboard.tsx:0016                Shield,
src/pages/Dashboard.tsx:0017                Terminal,
src/pages/Dashboard.tsx:0018                AlertTriangle,
src/pages/Dashboard.tsx:0019                Zap,
src/pages/Dashboard.tsx:0020            } from 'lucide-react';
src/pages/Dashboard.tsx:0021            import {
src/pages/Dashboard.tsx:0022                AdminApiKeyRecord,
src/pages/Dashboard.tsx:0023                fetchAdminApiKeys,
src/pages/Dashboard.tsx:0024                fetchCurrentUserSession,
src/pages/Dashboard.tsx:0025                fetchDashboardRulesSummary,
src/pages/Dashboard.tsx:0026                fetchEdgeAutoModules,
```

frontend/src/pages/Dashboard.tsx:27-76

```
src/pages/Dashboard.tsx:0027                fetchEdgeAutoSummary,
src/pages/Dashboard.tsx:0028                fetchLogs,
src/pages/Dashboard.tsx:0029                fetchStats,
src/pages/Dashboard.tsx:0030                getErrorMessage,
src/pages/Dashboard.tsx:0031                isRequestAbortError,
src/pages/Dashboard.tsx:0032            } from '../services/api';
src/pages/Dashboard.tsx:0033            import type { EdgeAutoModuleManifest, EdgeAutoSummary } from '../types/edge-auto';
src/pages/Dashboard.tsx:0034            import { EdgeAutoProtocolSummaryGrid } from '../components/EdgeAutoProtocolModules';
src/pages/Dashboard.tsx:0035            import { DashboardSecuritySummary, DashboardStats, LogItem } from '../types/api';
src/pages/Dashboard.tsx:0036            import ErrorDisplay from '../components/ErrorDisplay';
src/pages/Dashboard.tsx:0037            import { useWebSocket } from '../context/WebSocketContext';
src/pages/Dashboard.tsx:0038            import { useLiveMode } from '../context/LiveModeContext';
src/pages/Dashboard.tsx:0039            import { ConsoleEmptyState, ConsolePage, ConsolePanel, ConsoleStatCard } from '../components/ConsolePrimitives';
src/pages/Dashboard.tsx:0040            import StableResponsiveChart from '../components/StableResponsiveChart';
src/pages/Dashboard.tsx:0041
src/pages/Dashboard.tsx:0042            const defaultSecuritySummary: DashboardSecuritySummary = {
src/pages/Dashboard.tsx:0043                openCriticalFindings: 0,
src/pages/Dashboard.tsx:0044                highRiskFindings: 0,
src/pages/Dashboard.tsx:0045                activeAlerts: 0,
src/pages/Dashboard.tsx:0046                runningScans: 0,
src/pages/Dashboard.tsx:0047                queuedScans: 0,
src/pages/Dashboard.tsx:0048                pendingActions: 0,
src/pages/Dashboard.tsx:0049                failedValidations: 0,
src/pages/Dashboard.tsx:0050                assetsAtRisk: 0,
src/pages/Dashboard.tsx:0051                observedSessions: 0,
src/pages/Dashboard.tsx:0052                trafficAnomalies: 0,
src/pages/Dashboard.tsx:0053                recentActions: [],
src/pages/Dashboard.tsx:0054                recentValidations: [],
src/pages/Dashboard.tsx:0055            };
src/pages/Dashboard.tsx:0056
src/pages/Dashboard.tsx:0057            const TrafficChart = ({ data }: { data: { time: string; requests: number; blocked: number }[] }) => (
src/pages/Dashboard.tsx:0058                <StableResponsiveChart minHeight={300} loadingText="Waiting for chart container...">
src/pages/Dashboard.tsx:0059                    <AreaChart data={data}>
src/pages/Dashboard.tsx:0060                        <defs>
src/pages/Dashboard.tsx:0061                            <linearGradient id="colorRequests" x1="0" y1="0" x2="0" y2="1">
src/pages/Dashboard.tsx:0062                                <stop offset="5%" stopColor="#3b82f6" stopOpacity={0.18} />
src/pages/Dashboard.tsx:0063                                <stop offset="95%" stopColor="#3b82f6" stopOpacity={0} />
src/pages/Dashboard.tsx:0064                            </linearGradient>
src/pages/Dashboard.tsx:0065                            <linearGradient id="colorBlocked" x1="0" y1="0" x2="0" y2="1">
src/pages/Dashboard.tsx:0066                                <stop offset="5%" stopColor="#ef4444" stopOpacity={0.18} />
src/pages/Dashboard.tsx:0067                                <stop offset="95%" stopColor="#ef4444" stopOpacity={0} />
src/pages/Dashboard.tsx:0068                            </linearGradient>
src/pages/Dashboard.tsx:0069                        </defs>
src/pages/Dashboard.tsx:0070                        <CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" vertical={false} />
src/pages/Dashboard.tsx:0071                        <XAxis dataKey="time" stroke="hsl(var(--muted-foreground))" fontSize={11} tickLine={false} axisLine={false} dy={10} />
src/pages/Dashboard.tsx:0072                        <YAxis stroke="hsl(var(--muted-foreground))" fontSize={11} tickLine={false} axisLine={false} dx={-10} />
src/pages/Dashboard.tsx:0073                        <Tooltip
src/pages/Dashboard.tsx:0074                            contentStyle={{
src/pages/Dashboard.tsx:0075                                backgroundColor: 'hsl(var(--card))',
src/pages/Dashboard.tsx:0076                                borderColor: 'hsl(var(--border))',
```

frontend/src/pages/Dashboard.tsx:77-126

```
src/pages/Dashboard.tsx:0077                                borderRadius: '12px',
src/pages/Dashboard.tsx:0078                            }}
src/pages/Dashboard.tsx:0079                        />
src/pages/Dashboard.tsx:0080                        <Area type="monotone" dataKey="requests" stroke="#3b82f6" strokeWidth={2.5} fillOpacity={1} fill="url(#colorRequests)" />
src/pages/Dashboard.tsx:0081                        <Area type="monotone" dataKey="blocked" stroke="#ef4444" strokeWidth={2.5} fillOpacity={1} fill="url(#colorBlocked)" />
src/pages/Dashboard.tsx:0082                    </AreaChart>
src/pages/Dashboard.tsx:0083                </StableResponsiveChart>
src/pages/Dashboard.tsx:0084            );
src/pages/Dashboard.tsx:0085
src/pages/Dashboard.tsx:0086            const LogStream = ({ logs, emptyText }: { logs: LogItem[]; emptyText: string }) => (
src/pages/Dashboard.tsx:0087                <div className="h-full rounded-xl border border-border bg-muted/15 p-4">
src/pages/Dashboard.tsx:0088                    <div className="h-full space-y-2 overflow-y-auto font-mono text-xs">
src/pages/Dashboard.tsx:0089                        <AnimatePresence>
src/pages/Dashboard.tsx:0090                            {logs.map((log) => (
src/pages/Dashboard.tsx:0091                                <motion.div
src/pages/Dashboard.tsx:0092                                    key={`${log.id}-${log.time}`}
src/pages/Dashboard.tsx:0093                                    initial={{ opacity: 0, x: -12 }}
src/pages/Dashboard.tsx:0094                                    animate={{ opacity: 1, x: 0 }}
src/pages/Dashboard.tsx:0095                                    className={`flex gap-4 rounded-lg border p-2.5 ${
src/pages/Dashboard.tsx:0096                                        log.level === 'ERROR' || log.status === 403
src/pages/Dashboard.tsx:0097                                            ? 'border-rose-500/20 bg-rose-500/10 text-rose-600 dark:text-rose-300'
src/pages/Dashboard.tsx:0098                                            : 'border-border bg-card text-foreground'
src/pages/Dashboard.tsx:0099                                    }`}
src/pages/Dashboard.tsx:0100                                >
src/pages/Dashboard.tsx:0101                                    <span className="w-20 shrink-0 text-muted-foreground">{new Date(log.time).toLocaleTimeString()}</span>
src/pages/Dashboard.tsx:0102                                    <span className="w-12 shrink-0 font-bold">{log.status || 200}</span>
src/pages/Dashboard.tsx:0103                                    <span className="w-12 shrink-0 font-semibold text-primary">{log.type}</span>
src/pages/Dashboard.tsx:0104                                    <span className="break-all">{log.message}</span>
src/pages/Dashboard.tsx:0105                                </motion.div>
src/pages/Dashboard.tsx:0106                            ))}
src/pages/Dashboard.tsx:0107                        </AnimatePresence>
src/pages/Dashboard.tsx:0108                        {logs.length === 0 && <div className="py-8 text-center font-sans text-muted-foreground">{emptyText}</div>}
src/pages/Dashboard.tsx:0109                    </div>
src/pages/Dashboard.tsx:0110                </div>
src/pages/Dashboard.tsx:0111            );
src/pages/Dashboard.tsx:0112
src/pages/Dashboard.tsx:0113            const countryColor = (code: string) => ({
src/pages/Dashboard.tsx:0114                CN: 'bg-rose-500',
src/pages/Dashboard.tsx:0115                US: 'bg-blue-500',
src/pages/Dashboard.tsx:0116                RU: 'bg-rose-600',
src/pages/Dashboard.tsx:0117                JP: 'bg-amber-500',
src/pages/Dashboard.tsx:0118                DE: 'bg-cyan-500',
src/pages/Dashboard.tsx:0119                GB: 'bg-indigo-500',
src/pages/Dashboard.tsx:0120                Unknown: 'bg-slate-400',
src/pages/Dashboard.tsx:0121            }[code] || 'bg-slate-400');
src/pages/Dashboard.tsx:0122
src/pages/Dashboard.tsx:0123            type DashboardRuleSummary = Awaited<ReturnType<typeof fetchDashboardRulesSummary>>;
src/pages/Dashboard.tsx:0124
src/pages/Dashboard.tsx:0125            export default function Dashboard() {
src/pages/Dashboard.tsx:0126                const { subscribe } = useWebSocket();
```

frontend/src/pages/Dashboard.tsx:127-176

```
src/pages/Dashboard.tsx:0127                const { liveModeEnabled } = useLiveMode();
src/pages/Dashboard.tsx:0128                const [stats, setStats] = useState<DashboardStats | null>(null);
src/pages/Dashboard.tsx:0129                const [logs, setLogs] = useState<LogItem[]>([]);
src/pages/Dashboard.tsx:0130                const [error, setError] = useState<string | null>(null);
src/pages/Dashboard.tsx:0131                const [rulesSummary, setRulesSummary] = useState<DashboardRuleSummary>({ totalRules: 0, activeRules: 0, topRules: [] });
src/pages/Dashboard.tsx:0132                const [edgeAutoSummary, setEdgeAutoSummary] = useState<EdgeAutoSummary | null>(null);
src/pages/Dashboard.tsx:0133                const [edgeAutoModules, setEdgeAutoModules] = useState<EdgeAutoModuleManifest[]>([]);
src/pages/Dashboard.tsx:0134                const [apiKeys, setApiKeys] = useState<AdminApiKeyRecord[]>([]);
src/pages/Dashboard.tsx:0135                const [userRole, setUserRole] = useState('viewer');
src/pages/Dashboard.tsx:0136                const [timeRange, setTimeRange] = useState<'1h' | '24h'>('1h');
src/pages/Dashboard.tsx:0137                const [filterQuery, setFilterQuery] = useState('');
src/pages/Dashboard.tsx:0138                const [paused, setPaused] = useState(false);
src/pages/Dashboard.tsx:0139                const [exportMessage, setExportMessage] = useState<string | null>(null);
src/pages/Dashboard.tsx:0140
src/pages/Dashboard.tsx:0141                const filteredLogs = useMemo(() => logs.filter((log) => {
src/pages/Dashboard.tsx:0142                    if (!filterQuery.trim()) return true;
src/pages/Dashboard.tsx:0143                    const query = filterQuery.toLowerCase();
src/pages/Dashboard.tsx:0144                    return [log.type, log.level, log.message, log.path, log.method, String(log.status || '')]
src/pages/Dashboard.tsx:0145                        .filter(Boolean)
src/pages/Dashboard.tsx:0146                        .some((value) => String(value).toLowerCase().includes(query));
src/pages/Dashboard.tsx:0147                }), [filterQuery, logs]);
src/pages/Dashboard.tsx:0148
src/pages/Dashboard.tsx:0149                const filteredHistory = useMemo(
src/pages/Dashboard.tsx:0150                    () => stats?.history?.slice(timeRange === '1h' ? -10 : 0) || [],
src/pages/Dashboard.tsx:0151                    [stats?.history, timeRange],
src/pages/Dashboard.tsx:0152                );
src/pages/Dashboard.tsx:0153
src/pages/Dashboard.tsx:0154                const securitySummary = stats?.securityOperations || defaultSecuritySummary;
src/pages/Dashboard.tsx:0155
src/pages/Dashboard.tsx:0156                useEffect(() => {
src/pages/Dashboard.tsx:0157                    let disposed = false;
src/pages/Dashboard.tsx:0158                    let controller: AbortController | null = null;
src/pages/Dashboard.tsx:0159
src/pages/Dashboard.tsx:0160                    const loadData = async () => {
src/pages/Dashboard.tsx:0161                        controller?.abort();
src/pages/Dashboard.tsx:0162                        controller = new AbortController();
src/pages/Dashboard.tsx:0163                        try {
src/pages/Dashboard.tsx:0164                            const statsData = await fetchStats(controller.signal);
src/pages/Dashboard.tsx:0165                            if (!disposed && statsData) {
src/pages/Dashboard.tsx:0166                                setStats(statsData);
src/pages/Dashboard.tsx:0167                                setError(null);
src/pages/Dashboard.tsx:0168                            }
src/pages/Dashboard.tsx:0169                        } catch (err) {
src/pages/Dashboard.tsx:0170                            if (!disposed && !isRequestAbortError(err, controller.signal)) {
src/pages/Dashboard.tsx:0171                                setError(getErrorMessage(err, 'FG-DASHBOARD-STATS'));
src/pages/Dashboard.tsx:0172                            }
src/pages/Dashboard.tsx:0173                        }
src/pages/Dashboard.tsx:0174                    };
src/pages/Dashboard.tsx:0175
src/pages/Dashboard.tsx:0176                    void loadData();
```

frontend/src/pages/Dashboard.tsx:177-226

```
src/pages/Dashboard.tsx:0177                    if (!liveModeEnabled || paused) {
src/pages/Dashboard.tsx:0178                        return () => {
src/pages/Dashboard.tsx:0179                            disposed = true;
src/pages/Dashboard.tsx:0180                            controller?.abort();
src/pages/Dashboard.tsx:0181                        };
src/pages/Dashboard.tsx:0182                    }
src/pages/Dashboard.tsx:0183
src/pages/Dashboard.tsx:0184                    const interval = setInterval(() => {
src/pages/Dashboard.tsx:0185                        void loadData();
src/pages/Dashboard.tsx:0186                    }, 4000);
src/pages/Dashboard.tsx:0187
src/pages/Dashboard.tsx:0188                    return () => {
src/pages/Dashboard.tsx:0189                        disposed = true;
src/pages/Dashboard.tsx:0190                        controller?.abort();
src/pages/Dashboard.tsx:0191                        clearInterval(interval);
src/pages/Dashboard.tsx:0192                    };
src/pages/Dashboard.tsx:0193                }, [liveModeEnabled, paused]);
src/pages/Dashboard.tsx:0194
src/pages/Dashboard.tsx:0195                useEffect(() => {
src/pages/Dashboard.tsx:0196                    let disposed = false;
src/pages/Dashboard.tsx:0197                    const controller = new AbortController();
src/pages/Dashboard.tsx:0198
src/pages/Dashboard.tsx:0199                    void fetchLogs(controller.signal)
src/pages/Dashboard.tsx:0200                        .then((rows) => {
src/pages/Dashboard.tsx:0201                            if (!disposed && Array.isArray(rows)) {
src/pages/Dashboard.tsx:0202                                setLogs(rows);
src/pages/Dashboard.tsx:0203                            }
src/pages/Dashboard.tsx:0204                        })
src/pages/Dashboard.tsx:0205                        .catch((err) => {
src/pages/Dashboard.tsx:0206                            if (!disposed && !isRequestAbortError(err, controller.signal)) {
src/pages/Dashboard.tsx:0207                                setError(getErrorMessage(err, 'FG-DASHBOARD-LOGS'));
src/pages/Dashboard.tsx:0208                            }
src/pages/Dashboard.tsx:0209                        });
src/pages/Dashboard.tsx:0210
src/pages/Dashboard.tsx:0211                    const unsubscribe = subscribe<LogItem>('agent:log', (logData) => {
src/pages/Dashboard.tsx:0212                        if (paused) return;
src/pages/Dashboard.tsx:0213                        setLogs((prev) => [logData, ...prev].slice(0, 100));
src/pages/Dashboard.tsx:0214                    });
src/pages/Dashboard.tsx:0215
src/pages/Dashboard.tsx:0216                    return () => {
src/pages/Dashboard.tsx:0217                        disposed = true;
src/pages/Dashboard.tsx:0218                        controller.abort();
src/pages/Dashboard.tsx:0219                        unsubscribe();
src/pages/Dashboard.tsx:0220                    };
src/pages/Dashboard.tsx:0221                }, [paused, subscribe]);
src/pages/Dashboard.tsx:0222
src/pages/Dashboard.tsx:0223                useEffect(() => {
src/pages/Dashboard.tsx:0224                    let disposed = false;
src/pages/Dashboard.tsx:0225                    const controller = new AbortController();
src/pages/Dashboard.tsx:0226
```

frontend/src/pages/Dashboard.tsx:227-276

```
src/pages/Dashboard.tsx:0227                    const loadSideData = async () => {
src/pages/Dashboard.tsx:0228                        try {
src/pages/Dashboard.tsx:0229                            const user = await fetchCurrentUserSession(controller.signal);
src/pages/Dashboard.tsx:0230                            const role = user?.role || 'viewer';
src/pages/Dashboard.tsx:0231                            if (disposed) return;
src/pages/Dashboard.tsx:0232                            setUserRole(role);
src/pages/Dashboard.tsx:0233
src/pages/Dashboard.tsx:0234                            const [rules, edgeSummary, moduleManifests, keys] = await Promise.all([
src/pages/Dashboard.tsx:0235                                fetchDashboardRulesSummary(controller.signal),
src/pages/Dashboard.tsx:0236                                fetchEdgeAutoSummary(controller.signal).catch(() => null),
src/pages/Dashboard.tsx:0237                                fetchEdgeAutoModules(controller.signal).catch(() => []),
src/pages/Dashboard.tsx:0238                                role === 'admin' ? fetchAdminApiKeys(controller.signal).catch(() => []) : Promise.resolve([]),
src/pages/Dashboard.tsx:0239                            ]);
src/pages/Dashboard.tsx:0240
src/pages/Dashboard.tsx:0241                            if (disposed) return;
src/pages/Dashboard.tsx:0242                            setRulesSummary(rules || { totalRules: 0, topRules: [] });
src/pages/Dashboard.tsx:0243                            setEdgeAutoSummary(edgeSummary);
src/pages/Dashboard.tsx:0244                            setEdgeAutoModules(moduleManifests || []);
src/pages/Dashboard.tsx:0245                            setApiKeys(keys || []);
src/pages/Dashboard.tsx:0246                        } catch (err) {
src/pages/Dashboard.tsx:0247                            if (!disposed && !isRequestAbortError(err, controller.signal)) {
src/pages/Dashboard.tsx:0248                                setError(getErrorMessage(err, 'FG-DASHBOARD-SIDE'));
src/pages/Dashboard.tsx:0249                            }
src/pages/Dashboard.tsx:0250                        }
src/pages/Dashboard.tsx:0251                    };
src/pages/Dashboard.tsx:0252
src/pages/Dashboard.tsx:0253                    void loadSideData();
src/pages/Dashboard.tsx:0254
src/pages/Dashboard.tsx:0255                    return () => {
src/pages/Dashboard.tsx:0256                        disposed = true;
src/pages/Dashboard.tsx:0257                        controller.abort();
src/pages/Dashboard.tsx:0258                    };
src/pages/Dashboard.tsx:0259                }, []);
src/pages/Dashboard.tsx:0260
src/pages/Dashboard.tsx:0261                if (error) {
src/pages/Dashboard.tsx:0262                    return <ErrorDisplay title="Dashboard unavailable" message={error} onRetry={() => window.location.reload()} />;
src/pages/Dashboard.tsx:0263                }
src/pages/Dashboard.tsx:0264
src/pages/Dashboard.tsx:0265                if (!stats) {
src/pages/Dashboard.tsx:0266                    return (
src/pages/Dashboard.tsx:0267                        <div className="flex min-h-[400px] items-center justify-center">
src/pages/Dashboard.tsx:0268                            <motion.div
src/pages/Dashboard.tsx:0269                                animate={{ rotate: 360 }}
src/pages/Dashboard.tsx:0270                                transition={{ repeat: Infinity, duration: 1, ease: 'linear' }}
src/pages/Dashboard.tsx:0271                                className="h-8 w-8 rounded-full border-b-2 border-t-2 border-primary"
src/pages/Dashboard.tsx:0272                            />
src/pages/Dashboard.tsx:0273                        </div>
src/pages/Dashboard.tsx:0274                    );
src/pages/Dashboard.tsx:0275                }
src/pages/Dashboard.tsx:0276
```

frontend/src/pages/Dashboard.tsx:277-326

```
src/pages/Dashboard.tsx:0277                const exportSnapshot = (format: 'json' | 'csv') => {
src/pages/Dashboard.tsx:0278                    const payload = format === 'json'
src/pages/Dashboard.tsx:0279                        ? JSON.stringify({
src/pages/Dashboard.tsx:0280                            generatedAt: new Date().toISOString(),
src/pages/Dashboard.tsx:0281                            timeRange,
src/pages/Dashboard.tsx:0282                            filterQuery,
src/pages/Dashboard.tsx:0283                            stats,
src/pages/Dashboard.tsx:0284                            logs: filteredLogs,
src/pages/Dashboard.tsx:0285                        }, null, 2)
src/pages/Dashboard.tsx:0286                        : [
src/pages/Dashboard.tsx:0287                            'time,type,level,status,method,path,message',
src/pages/Dashboard.tsx:0288                            ...filteredLogs.map((log) => [
src/pages/Dashboard.tsx:0289                                log.time,
src/pages/Dashboard.tsx:0290                                log.type,
src/pages/Dashboard.tsx:0291                                log.level,
src/pages/Dashboard.tsx:0292                                log.status || '',
src/pages/Dashboard.tsx:0293                                log.method || '',
src/pages/Dashboard.tsx:0294                                log.path || '',
src/pages/Dashboard.tsx:0295                                `"${String(log.message).replace(/"/g, '""')}"`,
src/pages/Dashboard.tsx:0296                            ].join(',')),
src/pages/Dashboard.tsx:0297                        ].join('\n');
src/pages/Dashboard.tsx:0298
src/pages/Dashboard.tsx:0299                    const blob = new Blob([payload], { type: format === 'json' ? 'application/json' : 'text/csv;charset=utf-8' });
src/pages/Dashboard.tsx:0300                    const url = window.URL.createObjectURL(blob);
src/pages/Dashboard.tsx:0301                    const anchor = document.createElement('a');
src/pages/Dashboard.tsx:0302                    anchor.href = url;
src/pages/Dashboard.tsx:0303                    anchor.download = `dashboard-${timeRange}-${Date.now()}.${format}`;
src/pages/Dashboard.tsx:0304                    anchor.click();
src/pages/Dashboard.tsx:0305                    window.URL.revokeObjectURL(url);
src/pages/Dashboard.tsx:0306                    setExportMessage(`已导出当前视图 (${format.toUpperCase()}) `);
src/pages/Dashboard.tsx:0307                    window.setTimeout(() => setExportMessage(null), 2500);
src/pages/Dashboard.tsx:0308                };
src/pages/Dashboard.tsx:0309
src/pages/Dashboard.tsx:0310                const healthStatus = stats.successRate >= 99 ? '系统健康' : stats.successRate >= 95 ? '运行稳定' : '需要重点关注';
src/pages/Dashboard.tsx:0311                const emptyAttackDescription = securitySummary.observedSessions || securitySummary.trafficAnomalies
src/pages/Dashboard.tsx:0312                    ? `最近已观测 ${securitySummary.observedSessions || 0} 条会话，识别 ${securitySummary.trafficAnomalies || 0} 个异常流量特征。`
src/pages/Dashboard.tsx:0313                    : '当前没有新的来源分布，但系统仍在持续接收实时流量摘要。';
src/pages/Dashboard.tsx:0314                const emptyRulesDescription = securitySummary.activeAlerts || securitySummary.highRiskFindings
src/pages/Dashboard.tsx:0315                    ? `当前有 ${securitySummary.activeAlerts} 条活跃告警、${securitySummary.highRiskFindings || 0} 条高风险发现等待规则命中。`
src/pages/Dashboard.tsx:0316                    : '规则命中后这里会显示排行；当前可转到监控、告警和设备页查看实时状态。';
src/pages/Dashboard.tsx:0317                const emptyLogsText = securitySummary.pendingActions || securitySummary.failedValidations
src/pages/Dashboard.tsx:0318                    ? `当前没有新的日志流；待执行动作 ${securitySummary.pendingActions}，失败验证 ${securitySummary.failedValidations}。`
src/pages/Dashboard.tsx:0319                    : '正在等待新的日志事件进入当前视图。';
src/pages/Dashboard.tsx:0320
src/pages/Dashboard.tsx:0321                return (
src/pages/Dashboard.tsx:0322                    <ConsolePage title="安全控制台" subtitle="统一监控、规则、日志与运营摘要">
src/pages/Dashboard.tsx:0323                        <div className="grid grid-cols-1 gap-4 md:grid-cols-2 xl:grid-cols-4">
src/pages/Dashboard.tsx:0324                            <ConsoleStatCard icon={Server} label="总请求数" value={stats.totalRequests.toLocaleString()} hint="整体流量处理规模" tone="primary" />
src/pages/Dashboard.tsx:0325                            <ConsoleStatCard icon={Shield} label="拦截威胁" value={stats.blockedRequests.toLocaleString()} hint="近期拦截事件累计" tone="danger" />
src/pages/Dashboard.tsx:0326                            <ConsoleStatCard icon={Zap} label="平均延迟" value={`${stats.avgProcessingTime || 0}ms`} hint={`P99 ${stats.p99Latency || 0}ms`} tone=
```

frontend/src/pages/Dashboard.tsx:327-376

```
src/pages/Dashboard.tsx:0327                        <ConsoleStatCard icon={Globe} label="全局健康" value={`${stats.successRate}%`} hint={healthStatus} tone="success" />
src/pages/Dashboard.tsx:0328                    </div>
src/pages/Dashboard.tsx:0329
src/pages/Dashboard.tsx:0330                    <div className="grid grid-cols-1 gap-4 md:grid-cols-2 xl:grid-cols-4">
src/pages/Dashboard.tsx:0331                        <ConsoleStatCard icon={Cpu} label="CPU 使用率" value={`${stats.cpuUsage?.toFixed(1) || 0}%`} hint="主机 CPU" tone="primary" />
src/pages/Dashboard.tsx:0332                        <ConsoleStatCard icon={HardDrive} label="内存占用" value={`${stats.memoryUsage?.toFixed(1) || 0}%`} hint="系统内存" tone="warning" />
src/pages/Dashboard.tsx:0333                        <ConsoleStatCard icon={Database} label="磁盘占用" value={`${stats.diskUsage?.toFixed(1) || 0}%`} hint="根分区使用率" tone="neutral" />
src/pages/Dashboard.tsx:0334                        <ConsoleStatCard icon={Activity} label="网络吞吐" value={`${stats.networkUsage?.toFixed(2) || 0} MB/s`} hint="实时 I/O 吞吐量" tone="su
src/pages/Dashboard.tsx:0335                    </div>
src/pages/Dashboard.tsx:0336
src/pages/Dashboard.tsx:0337                    <div className="grid grid-cols-1 gap-4 md:grid-cols-2 xl:grid-cols-4">
src/pages/Dashboard.tsx:0338                        <ConsoleStatCard icon={AlertTriangle} label="高危发现" value={securitySummary.openCriticalFindings} hint={`高风险总数 ${securitySummary
src/pages/Dashboard.tsx:0339                        <ConsoleStatCard icon={Shield} label="活跃告警" value={securitySummary.activeAlerts} hint={`待处理动作 ${securitySummary.pendingActions
src/pages/Dashboard.tsx:0340                        <ConsoleStatCard icon={Search} label="扫描任务" value={securitySummary.runningScans} hint={`排队中 ${securitySummary.queuedScans || 0}
src/pages/Dashboard.tsx:0341                        <ConsoleStatCard icon={Globe} label="观测会话" value={securitySummary.observedSessions || 0} hint={`异常流量 ${securitySummary.traffic
src/pages/Dashboard.tsx:0342                    </div>
src/pages/Dashboard.tsx:0343
src/pages/Dashboard.tsx:0344                    {edgeAutoSummary && (
src/pages/Dashboard.tsx:0345                        <div className="grid grid-cols-1 gap-4 md:grid-cols-2 xl:grid-cols-4">
src/pages/Dashboard.tsx:0346                            <ConsoleStatCard icon={Server} label="Edge + Vehicle" value={(edgeAutoSummary.edgeNodes + edgeAutoSummary.vehicleNodes).toLocaleS
src/pages/Dashboard.tsx:0347                            <ConsoleStatCard icon={Cpu} label="ARM64 Nodes" value={edgeAutoSummary.arm64Nodes} hint={`Cloud ${edgeAutoSummary.cloudNodes}`} t
src/pages/Dashboard.tsx:0348                            <ConsoleStatCard icon={Activity} label="Bypass Nodes" value={edgeAutoSummary.readOnlyBypassNodes} hint={`Protocol anomalies ${edg
src/pages/Dashboard.tsx:0349                            <ConsoleStatCard icon={Download} label="MQTT Saved" value={`${edgeAutoSummary.mqttSavedMb} MB`} hint={`${edgeAutoSummary.mqttSave
src/pages/Dashboard.tsx:0350                        </div>
src/pages/Dashboard.tsx:0351                    )}
src/pages/Dashboard.tsx:0352
src/pages/Dashboard.tsx:0353                    {edgeAutoSummary && edgeAutoModules.length > 0 && (
src/pages/Dashboard.tsx:0354                        <ConsolePanel
src/pages/Dashboard.tsx:0355                            title="Protocol Modules"
src/pages/Dashboard.tsx:0356                            subtitle="Manifest-driven protocol module activity across edge and vehicle nodes"
src/pages/Dashboard.tsx:0357                            icon={Shield}
src/pages/Dashboard.tsx:0358                            iconTone="primary"
src/pages/Dashboard.tsx:0359                        >
src/pages/Dashboard.tsx:0360                            <EdgeAutoProtocolSummaryGrid
src/pages/Dashboard.tsx:0361                                manifests={edgeAutoModules}
src/pages/Dashboard.tsx:0362                                stats={undefined}
src/pages/Dashboard.tsx:0363                                finops={undefined}
src/pages/Dashboard.tsx:0364                                summary={edgeAutoSummary.modules}
src/pages/Dashboard.tsx:0365                                compact
src/pages/Dashboard.tsx:0366                            />
src/pages/Dashboard.tsx:0367                        </ConsolePanel>
src/pages/Dashboard.tsx:0368                    )}
src/pages/Dashboard.tsx:0369
src/pages/Dashboard.tsx:0370                    <div className="grid grid-cols-1 gap-6 lg:grid-cols-12">
src/pages/Dashboard.tsx:0371                        <div className="lg:col-span-8">
src/pages/Dashboard.tsx:0372                            <ConsolePanel
src/pages/Dashboard.tsx:0373                                title="流量分析"
src/pages/Dashboard.tsx:0374                                subtitle="按时间窗口查看请求与拦截趋势"
src/pages/Dashboard.tsx:0375                                icon={Activity}
src/pages/Dashboard.tsx:0376                                iconTone="primary"
```

frontend/src/pages/Dashboard.tsx:377-426

```
src/pages/Dashboard.tsx:0377                        >
src/pages/Dashboard.tsx:0378                            <div className="flex items-center justify-end gap-2 pb-4">
src/pages/Dashboard.tsx:0379                                <button
src/pages/Dashboard.tsx:0380                                    onClick={() => setTimeRange('1h')}
src/pages/Dashboard.tsx:0381                                    className={`rounded-lg border px-4 py-1.5 text-xs font-semibold ${
src/pages/Dashboard.tsx:0382                                        timeRange === '1h'
src/pages/Dashboard.tsx:0383                                            ? 'border-border bg-card text-foreground'
src/pages/Dashboard.tsx:0384                                            : 'border-transparent text-muted-foreground hover:text-foreground'
src/pages/Dashboard.tsx:0385                                    }`}
src/pages/Dashboard.tsx:0386                                >
src/pages/Dashboard.tsx:0387                                    1 小时
src/pages/Dashboard.tsx:0388                                </button>
src/pages/Dashboard.tsx:0389                                <button
src/pages/Dashboard.tsx:0390                                    onClick={() => setTimeRange('24h')}
src/pages/Dashboard.tsx:0391                                    className={`rounded-lg border px-4 py-1.5 text-xs font-semibold ${
src/pages/Dashboard.tsx:0392                                        timeRange === '24h'
src/pages/Dashboard.tsx:0393                                            ? 'border-border bg-card text-foreground'
src/pages/Dashboard.tsx:0394                                            : 'border-transparent text-muted-foreground hover:text-foreground'
src/pages/Dashboard.tsx:0395                                    }`}
src/pages/Dashboard.tsx:0396                                >
src/pages/Dashboard.tsx:0397                                    24 小时
src/pages/Dashboard.tsx:0398                                </button>
src/pages/Dashboard.tsx:0399                            </div>
src/pages/Dashboard.tsx:0400                            <TrafficChart data={filteredHistory} />
src/pages/Dashboard.tsx:0401                        </ConsolePanel>
src/pages/Dashboard.tsx:0402                    </div>
src/pages/Dashboard.tsx:0403
src/pages/Dashboard.tsx:0404                    <div className="lg:col-span-4">
src/pages/Dashboard.tsx:0405                        <ConsolePanel title="攻击来源" subtitle="按地区聚合的最近来源分布" icon={Globe} iconTone="warning">
src/pages/Dashboard.tsx:0406                            <div className="space-y-5">
src/pages/Dashboard.tsx:0407                                {(stats.attackOrigins || []).length > 0 ? (
src/pages/Dashboard.tsx:0408                                    stats.attackOrigins!.map((item, index) => (
src/pages/Dashboard.tsx:0409                                        <div key={`${item.countryCode}-${index}`}>
src/pages/Dashboard.tsx:0410                                            <div className="mb-2 flex items-center justify-between text-sm">
src/pages/Dashboard.tsx:0411                                                <span className="flex items-center gap-2 font-medium text-foreground">
src/pages/Dashboard.tsx:0412                                                    <span className={`h-2 w-2 rounded-full ${countryColor(item.countryCode)}`} />
src/pages/Dashboard.tsx:0413                                                    {item.country}
src/pages/Dashboard.tsx:0414                                                    <span className="text-muted-foreground">({item.countryCode})</span>
src/pages/Dashboard.tsx:0415                                                </span>
src/pages/Dashboard.tsx:0416                                                <span className="font-mono font-bold text-foreground">{Math.round(item.percentage)}%</span>
src/pages/Dashboard.tsx:0417                                            </div>
src/pages/Dashboard.tsx:0418                                            <div className="h-2 overflow-hidden rounded-full border border-border/50 bg-muted">
src/pages/Dashboard.tsx:0419                                                <motion.div
src/pages/Dashboard.tsx:0420                                                    initial={{ width: 0 }}
src/pages/Dashboard.tsx:0421                                                    animate={{ width: `${item.percentage}%` }}
src/pages/Dashboard.tsx:0422                                                    transition={{ duration: 0.6, delay: index * 0.06 }}
src/pages/Dashboard.tsx:0423                                                    className={`h-full ${countryColor(item.countryCode)}`}
src/pages/Dashboard.tsx:0424                                                />
src/pages/Dashboard.tsx:0425                                            </div>
src/pages/Dashboard.tsx:0426                                        </div>
```

frontend/src/pages/Dashboard.tsx:427-476

```
src/pages/Dashboard.tsx:0427                                        ))
src/pages/Dashboard.tsx:0428                                    ) : (
src/pages/Dashboard.tsx:0429                                        <ConsoleEmptyState title="暂无攻击来源分布" description={emptyAttackDescription} />
src/pages/Dashboard.tsx:0430                                    )}
src/pages/Dashboard.tsx:0431                                </div>
src/pages/Dashboard.tsx:0432                            </ConsolePanel>
src/pages/Dashboard.tsx:0433                        </div>
src/pages/Dashboard.tsx:0434                    </div>
src/pages/Dashboard.tsx:0435
src/pages/Dashboard.tsx:0436                    <div className="grid grid-cols-1 gap-6 lg:grid-cols-12">
src/pages/Dashboard.tsx:0437                        <div className="lg:col-span-8">
src/pages/Dashboard.tsx:0438                            <ConsolePanel
src/pages/Dashboard.tsx:0439                                title="实时日志流"
src/pages/Dashboard.tsx:0440                                subtitle="筛选当前视图并导出结果"
src/pages/Dashboard.tsx:0441                                icon={Terminal}
src/pages/Dashboard.tsx:0442                                iconTone="neutral"
src/pages/Dashboard.tsx:0443                                headerAction={(
src/pages/Dashboard.tsx:0444                                    <div className="flex items-center gap-3">
src/pages/Dashboard.tsx:0445                                        <div className="relative">
src/pages/Dashboard.tsx:0446                                            <Search className="absolute left-3 top-2 text-muted-foreground" size={16} />
src/pages/Dashboard.tsx:0447                                            <input
src/pages/Dashboard.tsx:0448                                                type="text"
src/pages/Dashboard.tsx:0449                                                value={filterQuery}
src/pages/Dashboard.tsx:0450                                                onChange={(event) => setFilterQuery(event.target.value)}
src/pages/Dashboard.tsx:0451                                                placeholder="筛选: status / type / path / message"
src/pages/Dashboard.tsx:0452                                                className="w-full rounded-lg border border-border bg-card py-1.5 pl-9 pr-3 text-sm text-foreground placeholde
src/pages/Dashboard.tsx:0453                                            />
src/pages/Dashboard.tsx:0454                                        </div>
src/pages/Dashboard.tsx:0455                                        <button
src/pages/Dashboard.tsx:0456                                            onClick={() => setPaused((current) => !current)}
src/pages/Dashboard.tsx:0457                                            className={`rounded-lg border border-border bg-card p-2 transition-colors hover:bg-muted/30 ${
src/pages/Dashboard.tsx:0458                                                paused ? 'text-amber-600 dark:text-amber-400' : 'text-muted-foreground hover:text-foreground'
src/pages/Dashboard.tsx:0459                                            }`}
src/pages/Dashboard.tsx:0460                                        >
src/pages/Dashboard.tsx:0461                                            <Pause size={18} />
src/pages/Dashboard.tsx:0462                                        </button>
src/pages/Dashboard.tsx:0463                                        <button
src/pages/Dashboard.tsx:0464                                            onClick={() => exportSnapshot('json')}
src/pages/Dashboard.tsx:0465                                            className="rounded-lg border border-border bg-card p-2 text-muted-foreground transition-colors hover:bg-muted/30
src/pages/Dashboard.tsx:0466                                        >
src/pages/Dashboard.tsx:0467                                            <Download size={18} />
src/pages/Dashboard.tsx:0468                                        </button>
src/pages/Dashboard.tsx:0469                                        <button
src/pages/Dashboard.tsx:0470                                            onClick={() => exportSnapshot('csv')}
src/pages/Dashboard.tsx:0471                                            className="rounded-lg border border-border bg-card px-3 py-2 text-xs font-medium text-muted-foreground transition
src/pages/Dashboard.tsx:0472                                        >
src/pages/Dashboard.tsx:0473                                            CSV
src/pages/Dashboard.tsx:0474                                        </button>
src/pages/Dashboard.tsx:0475                                    </div>
src/pages/Dashboard.tsx:0476                                )}
```

frontend/src/pages/Dashboard.tsx:477-526

```
src/pages/Dashboard.tsx:0477                        >
src/pages/Dashboard.tsx:0478                            {exportMessage && (
src/pages/Dashboard.tsx:0479                                <div className="mb-3 rounded-lg border border-emerald-500/20 bg-emerald-500/10 px-3 py-2 text-xs text-emerald-600 dark:te
src/pages/Dashboard.tsx:0480                                    {exportMessage}
src/pages/Dashboard.tsx:0481                                </div>
src/pages/Dashboard.tsx:0482                            )}
src/pages/Dashboard.tsx:0483                            <div className="h-[450px]">
src/pages/Dashboard.tsx:0484                                <LogStream logs={filteredLogs} emptyText={emptyLogsText} />
src/pages/Dashboard.tsx:0485                            </div>
src/pages/Dashboard.tsx:0486                        </ConsolePanel>
src/pages/Dashboard.tsx:0487                    </div>
src/pages/Dashboard.tsx:0488
src/pages/Dashboard.tsx:0489                    <div className="space-y-6 lg:col-span-4">
src/pages/Dashboard.tsx:0490                        <ConsolePanel title="最近运营动作" subtitle="来自扫描、处置和验证链路的最新结果" icon={Zap} iconTone="success">
src/pages/Dashboard.tsx:0491                            {((securitySummary.recentActions || []).length > 0 || (securitySummary.recentValidations || []).length > 0 ? (
src/pages/Dashboard.tsx:0492                                <div className="space-y-3">
src/pages/Dashboard.tsx:0493                                    {(securitySummary.recentActions || []).slice(0, 3).map((action) => (
src/pages/Dashboard.tsx:0494                                        <div key={action.actionId} className="rounded-xl border border-border bg-muted/15 p-3">
src/pages/Dashboard.tsx:0495                                            <div className="text-sm font-semibold text-foreground">{action.title}</div>
src/pages/Dashboard.tsx:0496                                            <div className="mt-1 flex items-center justify-between text-xs text-muted-foreground">
src/pages/Dashboard.tsx:0497                                                <span>{action.status}</span>
src/pages/Dashboard.tsx:0498                                                <span>{new Date(action.updatedAt || action.createdAt).toLocaleString()}</span>
src/pages/Dashboard.tsx:0499                                            </div>
src/pages/Dashboard.tsx:0500                                        </div>
src/pages/Dashboard.tsx:0501                                    ))}
src/pages/Dashboard.tsx:0502                                    {(securitySummary.recentValidations || []).slice(0, 2).map((validation) => (
src/pages/Dashboard.tsx:0503                                        <div key={validation.validationRunId} className="rounded-xl border border-border bg-muted/15 p-3">
src/pages/Dashboard.tsx:0504                                            <div className="text-sm font-semibold text-foreground">{validation.attackName}</div>
src/pages/Dashboard.tsx:0505                                            <div className="mt-1 flex items-center justify-between text-xs text-muted-foreground">
src/pages/Dashboard.tsx:0506                                                <span>{validation.verdict}</span>
src/pages/Dashboard.tsx:0507                                                <span>{new Date(validation.completedAt).toLocaleString()}</span>
src/pages/Dashboard.tsx:0508                                            </div>
src/pages/Dashboard.tsx:0509                                        </div>
src/pages/Dashboard.tsx:0510                                    ))}
src/pages/Dashboard.tsx:0511                                </div>
src/pages/Dashboard.tsx:0512                            ) : (
src/pages/Dashboard.tsx:0513                                <ConsoleEmptyState title="暂无新的运营动作" description="当扫描、处置或攻击验证完成后，这里会展示最近的真实结果。" />
src/pages/Dashboard.tsx:0514                            )}
src/pages/Dashboard.tsx:0515                        </ConsolePanel>
src/pages/Dashboard.tsx:0516
src/pages/Dashboard.tsx:0517                        <ConsolePanel
src/pages/Dashboard.tsx:0518                            title="活跃规则"
src/pages/Dashboard.tsx:0519                            subtitle="触发次数最高的规则清单"
src/pages/Dashboard.tsx:0520                            icon={Code}
src/pages/Dashboard.tsx:0521                            iconTone="primary"
src/pages/Dashboard.tsx:0522                            headerAction={(
src/pages/Dashboard.tsx:0523                                <span className="rounded-full border border-border bg-muted px-3 py-1 text-xs font-semibold text-muted-foreground">
src/pages/Dashboard.tsx:0524                                    总计 {rulesSummary.totalRules || 0}
src/pages/Dashboard.tsx:0525                                </span>
src/pages/Dashboard.tsx:0526                            )}
```

frontend/src/pages/Dashboard.tsx:527-533 | frontend/src/pages/Devices.tsx:1-43

```
src/pages/Dashboard.tsx:0527                                >
src/pages/Dashboard.tsx:0528                                    <div className="space-y-4">
src/pages/Dashboard.tsx:0529                                        {(rulesSummary.topRules || []).length > 0 ? (
src/pages/Dashboard.tsx:0530                                            (rulesSummary.topRules || []).slice(0, 5).map((rule) => (
src/pages/Dashboard.tsx:0531                                                <div key={rule.id} className="flex items-center justify-between rounded-xl border border-border bg-muted/15 p-3 t
src/pages/Dashboard.tsx:0532                                                    <div className="flex items-center gap-4">
src/pages/Dashboard.tsx:0533                                                        <div className="flex h-10 w-10 items-center justify-center rounded-lg border border-primary/15 bg-primary
src/pages/Devices.tsx:0001              import { useEffect, useMemo, useState } from 'react';
src/pages/Devices.tsx:0002              import { Link } from 'react-router-dom';
src/pages/Devices.tsx:0003              import { useLanguage } from '../context/LanguageContext';
src/pages/Devices.tsx:0004              import { useLiveMode } from '../context/LiveModeContext';
src/pages/Devices.tsx:0005              import {
src/pages/Devices.tsx:0006                  Cpu, HardDrive, Wifi, WifiOff, Radio, RefreshCw, Terminal, Power, Settings, X,
src/pages/Devices.tsx:0007                  Server, Activity, Shield, Globe, Zap, ChevronRight, Save, RotateCcw, History, CheckCircle2
src/pages/Devices.tsx:0008              } from 'lucide-react';
src/pages/Devices.tsx:0009              import { fetchDeviceConfig, fetchDevicesList, fetchSystemBadges, getErrorMessage, isRequestAbortError, requestDevicePhysicalConfirmation, saveDeviceC
src/pages/Devices.tsx:0010
src/pages/Devices.tsx:0011              interface DeviceCommandRecord {
src/pages/Devices.tsx:0012                  id: string;
src/pages/Devices.tsx:0013                  deviceId: string;
src/pages/Devices.tsx:0014                  command: string;
src/pages/Devices.tsx:0015                  status: 'queued' | 'completed' | 'failed';
src/pages/Devices.tsx:0016                  message: string;
src/pages/Devices.tsx:0017                  executedBy: string;
src/pages/Devices.tsx:0018                  createdAt: string;
src/pages/Devices.tsx:0019                  completedAt?: string;
src/pages/Devices.tsx:0020              }
src/pages/Devices.tsx:0021
src/pages/Devices.tsx:0022              interface DeviceConfig {
src/pages/Devices.tsx:0023                  displayName: string;
src/pages/Devices.tsx:0024                  group: string;
src/pages/Devices.tsx:0025                  site: string;
src/pages/Devices.tsx:0026                  networkProfile: string;
src/pages/Devices.tsx:0027                  scanProfile: string;
src/pages/Devices.tsx:0028                  notes: string;
src/pages/Devices.tsx:0029                  commandPermissions: string[];
src/pages/Devices.tsx:0030                  tags: string[];
src/pages/Devices.tsx:0031                  updatedAt?: string;
src/pages/Devices.tsx:0032              }
src/pages/Devices.tsx:0033
src/pages/Devices.tsx:0034              interface Device {
src/pages/Devices.tsx:0035                  deviceId: string;
src/pages/Devices.tsx:0036                  version: string;
src/pages/Devices.tsx:0037                  chipModel: string;
src/pages/Devices.tsx:0038                  flashSize: number;
src/pages/Devices.tsx:0039                  freeHeap: number;
src/pages/Devices.tsx:0040                  sdkVersion: string;
src/pages/Devices.tsx:0041                  mode: 'online' | 'offline';
src/pages/Devices.tsx:0042                  isOnline: boolean;
src/pages/Devices.tsx:0043                  lastHeartbeat: string;
```

frontend/src/pages/Devices.tsx:44-93

```
src/pages/Devices.tsx:0044                  connectedAgents: string[];
src/pages/Devices.tsx:0045                  config?: DeviceConfig;
src/pages/Devices.tsx:0046                  latestCommand?: DeviceCommandRecord | null;
src/pages/Devices.tsx:0047                  status?: {
src/pages/Devices.tsx:0048                      cpuUsage: number;
src/pages/Devices.tsx:0049                      memoryUsage: number;
src/pages/Devices.tsx:0050                      uptime: number;
src/pages/Devices.tsx:0051                      wifiRssi: number;
src/pages/Devices.tsx:0052                  };
src/pages/Devices.tsx:0053                  systemInfo?: {
src/pages/Devices.tsx:0054                      hostname: string;
src/pages/Devices.tsx:0055                      os: string;
src/pages/Devices.tsx:0056                      kernel: string;
src/pages/Devices.tsx:0057                      arch: string;
src/pages/Devices.tsx:0058                  };
src/pages/Devices.tsx:0059                  transport?: 'wifi' | 'usb_serial' | 'ethernet' | 'unknown';
src/pages/Devices.tsx:0060                  externalPowerPresent?: boolean;
src/pages/Devices.tsx:0061                  secureElementPresent?: boolean;
src/pages/Devices.tsx:0062                  pendingPhysicalApproval?: boolean;
src/pages/Devices.tsx:0063                  interlockState?: 'disabled' | 'armed' | 'asserted' | 'fault' | 'unknown';
src/pages/Devices.tsx:0064                  hostActionState?: 'idle' | 'running' | 'awaiting confirmation' | 'completed' | 'failed' | 'unknown';
src/pages/Devices.tsx:0065                  physicalAuth?: {
src/pages/Devices.tsx:0066                      hasPublicKey?: boolean;
src/pages/Devices.tsx:0067                      monotonicCounter?: number;
src/pages/Devices.tsx:0068                      lastConfirmedAt?: string | null;
src/pages/Devices.tsx:0069                  };
src/pages/Devices.tsx:0070                  latestConfirmationSession?: {
src/pages/Devices.tsx:0071                      id: string;
src/pages/Devices.tsx:0072                      commandType: string;
src/pages/Devices.tsx:0073                      status: string;
src/pages/Devices.tsx:0074                      createdAt: string;
src/pages/Devices.tsx:0075                      confirmedAt?: string;
src/pages/Devices.tsx:0076                  } | null;
src/pages/Devices.tsx:0077                  securitySummary?: {
src/pages/Devices.tsx:0078                      asset?: {
src/pages/Devices.tsx:0079                          riskScore?: number;
src/pages/Devices.tsx:0080                          findingsOpen?: number;
src/pages/Devices.tsx:0081                          alertsOpen?: number;
src/pages/Devices.tsx:0082                      } | null;
src/pages/Devices.tsx:0083                      findings?: { title: string; severity: string }[];
src/pages/Devices.tsx:0084                      alerts?: { title: string; status: string }[];
src/pages/Devices.tsx:0085                      actions?: { title: string; status: string }[];
src/pages/Devices.tsx:0086                      latestCommand?: DeviceCommandRecord | null;
src/pages/Devices.tsx:0087                  };
src/pages/Devices.tsx:0088              }
src/pages/Devices.tsx:0089
src/pages/Devices.tsx:0090              const PHYSICAL_CONFIRMATION_COMMANDS = new Set([
src/pages/Devices.tsx:0091                  'reboot',
src/pages/Devices.tsx:0092                  'host_remove',
src/pages/Devices.tsx:0093                  'recovery_flash',
```

frontend/src/pages/Devices.tsx:94-143

```
src/pages/Devices.tsx:0094                'factory_reset',
src/pages/Devices.tsx:0095                'diagnostic_unlock',
src/pages/Devices.tsx:0096                'vehicle_ota_flash',
src/pages/Devices.tsx:0097                'cellular_breaker_assert',
src/pages/Devices.tsx:0098                'cellular_breaker_release'
src/pages/Devices.tsx:0099            ]);
src/pages/Devices.tsx:0100
src/pages/Devices.tsx:0101            const defaultConfig = (device: Device): DeviceConfig => ({
src/pages/Devices.tsx:0102                displayName: device.config?.displayName || device.deviceId,
src/pages/Devices.tsx:0103                group: device.config?.group || 'default',
src/pages/Devices.tsx:0104                site: device.config?.site || device.systemInfo?.hostname || 'unassigned',
src/pages/Devices.tsx:0105                networkProfile: device.config?.networkProfile || 'cloud-mesh',
src/pages/Devices.tsx:0106                scanProfile: device.config?.scanProfile || 'balanced',
src/pages/Devices.tsx:0107                notes: device.config?.notes || '',
src/pages/Devices.tsx:0108                commandPermissions: device.config?.commandPermissions || ['collect', 'sync-config', 'host_backup', 'host_install', 'reboot', 'host_remove', 'reco
src/pages/Devices.tsx:0109                tags: device.config?.tags || []
src/pages/Devices.tsx:0110            });
src/pages/Devices.tsx:0111
src/pages/Devices.tsx:0112            const formatDateTime = (value?: string) => (value ? new Date(value).toLocaleString() : '--');
src/pages/Devices.tsx:0113
src/pages/Devices.tsx:0114            const commandLabel = (command: string) => {
src/pages/Devices.tsx:0115                switch (command) {
src/pages/Devices.tsx:0116                    case 'collect':
src/pages/Devices.tsx:0117                        return 'Collect';
src/pages/Devices.tsx:0118                    case 'host_backup':
src/pages/Devices.tsx:0119                        return 'Backup';
src/pages/Devices.tsx:0120                    case 'host_remove':
src/pages/Devices.tsx:0121                        return 'Remove';
src/pages/Devices.tsx:0122                    case 'recovery_flash':
src/pages/Devices.tsx:0123                        return 'Recovery Flash';
src/pages/Devices.tsx:0124                    case 'factory reset':
src/pages/Devices.tsx:0125                        return 'Factory Reset';
src/pages/Devices.tsx:0126                    case 'host_install':
src/pages/Devices.tsx:0127                        return 'Install';
src/pages/Devices.tsx:0128                    case 'diagnostic_unlock':
src/pages/Devices.tsx:0129                        return 'Diagnostic Unlock';
src/pages/Devices.tsx:0130                    case 'vehicle_ota_flash':
src/pages/Devices.tsx:0131                        return 'Vehicle OTA';
src/pages/Devices.tsx:0132                    case 'cellular_breaker_assert':
src/pages/Devices.tsx:0133                        return 'Assert Breaker';
src/pages/Devices.tsx:0134                    case 'cellular_breaker_release':
src/pages/Devices.tsx:0135                        return 'Release Breaker';
src/pages/Devices.tsx:0136                    default:
src/pages/Devices.tsx:0137                        return command;
src/pages/Devices.tsx:0138                }
src/pages/Devices.tsx:0139            };
src/pages/Devices.tsx:0140
src/pages/Devices.tsx:0141            const DeviceCard = ({
src/pages/Devices.tsx:0142                device,
src/pages/Devices.tsx:0143                onCommand,
```

frontend/src/pages/Devices.tsx:144-193

```
src/pages/Devices.tsx:0144                  onOpenSettings,
src/pages/Devices.tsx:0145                  onSelect,
src/pages/Devices.tsx:0146                  isSelected,
src/pages/Devices.tsx:0147                  commandBusy
src/pages/Devices.tsx:0148              }: {
src/pages/Devices.tsx:0149                  device: Device;
src/pages/Devices.tsx:0150                  onCommand: (cmd: string) => void;
src/pages/Devices.tsx:0151                  onOpenSettings: () => void;
src/pages/Devices.tsx:0152                  onSelect: () => void;
src/pages/Devices.tsx:0153                  isSelected: boolean;
src/pages/Devices.tsx:0154                  commandBusy?: string | null;
src/pages/Devices.tsx:0155              }) => {
src/pages/Devices.tsx:0156                  const { t } = useLanguage();
src/pages/Devices.tsx:0157                  const latestCommand = device.latestCommand || device.securitySummary?.latestCommand || null;
src/pages/Devices.tsx:0158
src/pages/Devices.tsx:0159                  return (
src/pages/Devices.tsx:0160                      <div
src/pages/Devices.tsx:0161                          onClick={onSelect}
src/pages/Devices.tsx:0162                          className={`bg-card border rounded-xl p-5 shadow-sm transition-all cursor-pointer hover:shadow-md ${
src/pages/Devices.tsx:0163                              isSelected ? 'border-primary ring-2 ring-primary/20' : 'border-border hover:border-primary/30'
src/pages/Devices.tsx:0164                          }`}
src/pages/Devices.tsx:0165                      >
src/pages/Devices.tsx:0166                          <div className="flex justify-between items-start mb-4 gap-3">
src/pages/Devices.tsx:0167                              <div className="flex items-center gap-3">
src/pages/Devices.tsx:0168                                  <div className={`p-2.5 rounded-lg ${device.isOnline
src/pages/Devices.tsx:0169                                      ? device.mode === 'online'
src/pages/Devices.tsx:0170                                          ? 'bg-blue-500/10 text-blue-500'
src/pages/Devices.tsx:0171                                          : 'bg-emerald-500/10 text-emerald-500'
src/pages/Devices.tsx:0172                                      : 'bg-muted text-muted-foreground'}`}>
src/pages/Devices.tsx:0173                                      <Radio size={20} />
src/pages/Devices.tsx:0174                                  </div>
src/pages/Devices.tsx:0175                                  <div>
src/pages/Devices.tsx:0176                                      <div className="font-semibold text-foreground">{device.config?.displayName || device.deviceId}</div>
src/pages/Devices.tsx:0177                                      <div className="text-xs text-muted-foreground">{device.chipModel} v{device.version}</div>
src/pages/Devices.tsx:0178                                  </div>
src/pages/Devices.tsx:0179                              </div>
src/pages/Devices.tsx:0180
src/pages/Devices.tsx:0181                              <div className={`flex items-center gap-1.5 px-2 py-1 rounded-full text-xs font-medium ${device.isOnline
src/pages/Devices.tsx:0182                                  ? device.mode === 'online'
src/pages/Devices.tsx:0183                                      ? 'bg-blue-500/10 text-blue-500 border border-blue-500/20'
src/pages/Devices.tsx:0184                                      : 'bg-emerald-500/10 text-emerald-500 border border-emerald-500/20'
src/pages/Devices.tsx:0185                                  : 'bg-muted text-muted-foreground border border-border'}`}>
src/pages/Devices.tsx:0186                                  {device.isOnline ? (
src/pages/Devices.tsx:0187                                      device.mode === 'online' ? <><Wifi size={12} /> {t('devices.cloud_mode')}</> : <><WifiOff size={12} /> {t('devices.no_agents'
src/pages/Devices.tsx:0188                                  ) : (
src/pages/Devices.tsx:0189                                      <><span className="w-1.5 h-1.5 rounded-full bg-muted-foreground"></span> Disconnected</>
src/pages/Devices.tsx:0190                                  )}
src/pages/Devices.tsx:0191                              </div>
src/pages/Devices.tsx:0192                          </div>
src/pages/Devices.tsx:0193
```

frontend/src/pages/Devices.tsx:194-243

```
src/pages/Devices.tsx:0194                          {device.status && (
src/pages/Devices.tsx:0195                              <div className="grid grid-cols-3 gap-3 mb-4">
src/pages/Devices.tsx:0196                                  <div className="bg-muted/30 rounded-lg p-2 text-center">
src/pages/Devices.tsx:0197                                      <Cpu size={14} className="mx-auto mb-1 text-muted-foreground" />
src/pages/Devices.tsx:0198                                      <div className="text-sm font-medium text-foreground">{device.status.cpuUsage}%</div>
src/pages/Devices.tsx:0199                                      <div className="text-xxs text-muted-foreground">{t('devices.cpu')}</div>
src/pages/Devices.tsx:0200                                  </div>
src/pages/Devices.tsx:0201                                  <div className="bg-muted/30 rounded-lg p-2 text-center">
src/pages/Devices.tsx:0202                                      <HardDrive size={14} className="mx-auto mb-1 text-muted-foreground" />
src/pages/Devices.tsx:0203                                      <div className="text-sm font-medium text-foreground">{device.status.memoryUsage.toFixed(0)}%</div>
src/pages/Devices.tsx:0204                                      <div className="text-xxs text-muted-foreground">{t('devices.memory')}</div>
src/pages/Devices.tsx:0205                                  </div>
src/pages/Devices.tsx:0206                                  <div className="bg-muted/30 rounded-lg p-2 text-center">
src/pages/Devices.tsx:0207                                      <Activity size={14} className="mx-auto mb-1 text-muted-foreground" />
src/pages/Devices.tsx:0208                                      <div className="text-sm font-medium text-foreground">{device.status.wifiRssi}dBm</div>
src/pages/Devices.tsx:0209                                      <div className="text-xxs text-muted-foreground">{t('devices.signal')}</div>
src/pages/Devices.tsx:0210                                  </div>
src/pages/Devices.tsx:0211                              </div>
src/pages/Devices.tsx:0212                          )}
src/pages/Devices.tsx:0213
src/pages/Devices.tsx:0214                          <div className="mb-4 space-y-2 text-sm">
src/pages/Devices.tsx:0215                              <div className="flex items-center justify-between">
src/pages/Devices.tsx:0216                                  <span className="text-muted-foreground">{t('devices.connected_agents')}</span>
src/pages/Devices.tsx:0217                                  <span className="font-medium text-foreground flex items-center gap-1"><Server size={14} className="text-primary" />{device.connec
src/pages/Devices.tsx:0218                              </div>
src/pages/Devices.tsx:0219                              <div className="flex items-center justify-between">
src/pages/Devices.tsx:0220                                  <span className="text-muted-foreground">Risk</span>
src/pages/Devices.tsx:0221                                  <span className="font-medium text-foreground">{device.securitySummary?.asset?.riskScore || 0}</span>
src/pages/Devices.tsx:0222                              </div>
src/pages/Devices.tsx:0223                              <div className="text-xs text-muted-foreground">
src/pages/Devices.tsx:0224                                  Latest action: <span className="text-foreground">{latestCommand?.command || 'none'}</span>
src/pages/Devices.tsx:0225                              </div>
src/pages/Devices.tsx:0226                              <div className="text-xs text-muted-foreground">
src/pages/Devices.tsx:0227                                  Link: <span className="text-foreground">{device.transport || 'unknown'}</span> · Secure element: <span className="text-foreground
src/pages/Devices.tsx:0228                              </div>
src/pages/Devices.tsx:0229                              <div className="text-xs text-muted-foreground">
src/pages/Devices.tsx:0230                                  Power / Interlock: <span className="text-foreground">{device.externalPowerPresent ? 'external DC' : 'USB only'}</span> · <span cl
src/pages/Devices.tsx:0231                              </div>
src/pages/Devices.tsx:0232                          </div>
src/pages/Devices.tsx:0233
src/pages/Devices.tsx:0234                          <div className="flex gap-2">
src/pages/Devices.tsx:0235                              <button
src/pages/Devices.tsx:0236                                  onClick={(e) => { e.stopPropagation(); onCommand('collect'); }}
src/pages/Devices.tsx:0237                                  disabled={!device.isOnline || commandBusy === 'collect'}
src/pages/Devices.tsx:0238                                  className="flex-1 flex items-center justify-center gap-1.5 px-3 py-2 text-xs font-medium bg-primary text-primary-foreground round
src/pages/Devices.tsx:0239                              >
src/pages/Devices.tsx:0240                                  <Zap size={12} /> {commandBusy === 'collect' ? 'Running...' : 'Collect'}
src/pages/Devices.tsx:0241                              </button>
src/pages/Devices.tsx:0242                              <button
src/pages/Devices.tsx:0243                                  onClick={(e) => { e.stopPropagation(); onCommand('host_backup'); }}
```

frontend/src/pages/Devices.tsx:244-293

```
src/pages/Devices.tsx:0244                                  disabled={!device.isOnline || commandBusy === 'host_backup'}
src/pages/Devices.tsx:0245                                  className="flex items-center justify-center gap-1.5 px-3 py-2 text-xs font-medium bg-muted text-foreground rounded-lg hover:bg-mu
src/pages/Devices.tsx:0246                              >
src/pages/Devices.tsx:0247                                  <Save size={12} />
src/pages/Devices.tsx:0248                              </button>
src/pages/Devices.tsx:0249                              <button
src/pages/Devices.tsx:0250                                  onClick={(e) => { e.stopPropagation(); onCommand('reboot'); }}
src/pages/Devices.tsx:0251                                  disabled={!device.isOnline || commandBusy === 'reboot'}
src/pages/Devices.tsx:0252                                  className="flex items-center justify-center gap-1.5 px-3 py-2 text-xs font-medium bg-muted text-foreground rounded-lg hover:bg-mu
src/pages/Devices.tsx:0253                              >
src/pages/Devices.tsx:0254                                  <Power size={12} />
src/pages/Devices.tsx:0255                              </button>
src/pages/Devices.tsx:0256                              <button
src/pages/Devices.tsx:0257                                  onClick={(e) => { e.stopPropagation(); onOpenSettings(); }}
src/pages/Devices.tsx:0258                                  className="flex items-center justify-center gap-1.5 px-3 py-2 text-xs font-medium bg-muted text-foreground rounded-lg hover:bg-mu
src/pages/Devices.tsx:0259                              >
src/pages/Devices.tsx:0260                                  <Settings size={12} />
src/pages/Devices.tsx:0261                              </button>
src/pages/Devices.tsx:0262                              <div onClick={(e) => e.stopPropagation()} className="ml-auto">
src/pages/Devices.tsx:0263                                  <Link to={`/devices/board-status/${device.deviceId}`} className="px-3 py-2 text-xs font-medium bg-emerald-500/10 text-emerald-500
src/pages/Devices.tsx:0264                                      <Activity size={12} /> View Status
src/pages/Devices.tsx:0265                                  </Link>
src/pages/Devices.tsx:0266                              </div>
src/pages/Devices.tsx:0267                          </div>
src/pages/Devices.tsx:0268                      </div>
src/pages/Devices.tsx:0269                  );
src/pages/Devices.tsx:0270              };
src/pages/Devices.tsx:0271
src/pages/Devices.tsx:0272              const Devices = () => {
src/pages/Devices.tsx:0273                  const { t } = useLanguage();
src/pages/Devices.tsx:0274                  const { liveModeEnabled } = useLiveMode();
src/pages/Devices.tsx:0275                  const [devices, setDevices] = useState<Device[]>([]);
src/pages/Devices.tsx:0276                  const [loading, setLoading] = useState(true);
src/pages/Devices.tsx:0277                  const [selectedDevice, setSelectedDevice] = useState<string | null>(null);
src/pages/Devices.tsx:0278                  const [error, setError] = useState<string | null>(null);
src/pages/Devices.tsx:0279                  const [actionMessage, setActionMessage] = useState<string | null>(null);
src/pages/Devices.tsx:0280                  const [settingsOpen, setSettingsOpen] = useState(false);
src/pages/Devices.tsx:0281                  const [settingsDeviceId, setSettingsDeviceId] = useState<string | null>(null);
src/pages/Devices.tsx:0282                  const [configDraft, setConfigDraft] = useState<DeviceConfig | null>(null);
src/pages/Devices.tsx:0283                  const [configLoading, setConfigLoading] = useState(false);
src/pages/Devices.tsx:0284                  const [configSaving, setConfigSaving] = useState(false);
src/pages/Devices.tsx:0285                  const [commandHistory, setCommandHistory] = useState<DeviceCommandRecord[]>([]);
src/pages/Devices.tsx:0286                  const [commandBusyByDevice, setCommandBusyByDevice] = useState<Record<string, string | null>>({});
src/pages/Devices.tsx:0287                  const [platformBadges, setPlatformBadges] = useState({ agents: 0, alerts: 0, devices: 0, agentMonitor: 0, traffic: 0, scanner: 0 });
src/pages/Devices.tsx:0288
src/pages/Devices.tsx:0289                  const fetchDevices = async (signal?: AbortSignal) => {
src/pages/Devices.tsx:0290                      setLoading(true);
src/pages/Devices.tsx:0291                      setError(null);
src/pages/Devices.tsx:0292                      try {
src/pages/Devices.tsx:0293                          const [data, badges] = await Promise.all([
```

frontend/src/pages/Devices.tsx:294-343

```
src/pages/Devices.tsx:0294                              fetchDevicesList(signal),
src/pages/Devices.tsx:0295                              fetchSystemBadges(signal),
src/pages/Devices.tsx:0296                          ]);
src/pages/Devices.tsx:0297                          if (signal?.aborted) {
src/pages/Devices.tsx:0298                              return;
src/pages/Devices.tsx:0299                          }
src/pages/Devices.tsx:0300                          const nextDevices = data.devices || [];
src/pages/Devices.tsx:0301                          setDevices(nextDevices);
src/pages/Devices.tsx:0302                          setPlatformBadges(badges);
src/pages/Devices.tsx:0303                          setSelectedDevice((current) => current && nextDevices.some((device: Device) => device.deviceId === current) ? current : nextDevices[0]?.d
src/pages/Devices.tsx:0304                      } catch (e) {
src/pages/Devices.tsx:0305                          if (!isRequestAbortError(e, signal)) {
src/pages/Devices.tsx:0306                              setError(getErrorMessage(e, 'FG-DEVICES-LIST-ERR'));
src/pages/Devices.tsx:0307                              setDevices([]);
src/pages/Devices.tsx:0308                          }
src/pages/Devices.tsx:0309                      } finally {
src/pages/Devices.tsx:0310                          if (!signal?.aborted) {
src/pages/Devices.tsx:0311                              setLoading(false);
src/pages/Devices.tsx:0312                          }
src/pages/Devices.tsx:0313                      }
src/pages/Devices.tsx:0314                  };
src/pages/Devices.tsx:0315
src/pages/Devices.tsx:0316                  const loadDeviceConfig = async (deviceId: string) => {
src/pages/Devices.tsx:0317                      setConfigLoading(true);
src/pages/Devices.tsx:0318                      try {
src/pages/Devices.tsx:0319                          const response = await fetchDeviceConfig(deviceId);
src/pages/Devices.tsx:0320                          setConfigDraft(response?.config || null);
src/pages/Devices.tsx:0321                          setCommandHistory(response?.commandHistory || []);
src/pages/Devices.tsx:0322                          setError(null);
src/pages/Devices.tsx:0323                      } catch (e) {
src/pages/Devices.tsx:0324                          setError(getErrorMessage(e, 'FG-DEVICE-CONFIG-ERR'));
src/pages/Devices.tsx:0325                      } finally {
src/pages/Devices.tsx:0326                          setConfigLoading(false);
src/pages/Devices.tsx:0327                      }
src/pages/Devices.tsx:0328                  };
src/pages/Devices.tsx:0329
src/pages/Devices.tsx:0330                  useEffect(() => {
src/pages/Devices.tsx:0331                      const controller = new AbortController();
src/pages/Devices.tsx:0332                      void fetchDevices(controller.signal);
src/pages/Devices.tsx:0333                      return () => controller.abort();
src/pages/Devices.tsx:0334                  }, []);
src/pages/Devices.tsx:0335
src/pages/Devices.tsx:0336                  useEffect(() => {
src/pages/Devices.tsx:0337                      let disposed = false;
src/pages/Devices.tsx:0338                      let controller: AbortController | null = null;
src/pages/Devices.tsx:0339
src/pages/Devices.tsx:0340                      if (!liveModeEnabled) {
src/pages/Devices.tsx:0341                          return;
src/pages/Devices.tsx:0342                      }
src/pages/Devices.tsx:0343
```

frontend/src/pages/Devices.tsx:344-393

```
src/pages/Devices.tsx:0344                    const refresh = async () => {
src/pages/Devices.tsx:0345                        const nextController = new AbortController();
src/pages/Devices.tsx:0346                        controller?.abort();
src/pages/Devices.tsx:0347                        controller = nextController;
src/pages/Devices.tsx:0348                        await fetchDevices(nextController.signal);
src/pages/Devices.tsx:0349                    };
src/pages/Devices.tsx:0350
src/pages/Devices.tsx:0351                    const interval = setInterval(() => {
src/pages/Devices.tsx:0352                        if (!disposed) {
src/pages/Devices.tsx:0353                            void refresh();
src/pages/Devices.tsx:0354                        }
src/pages/Devices.tsx:0355                    }, 10000);
src/pages/Devices.tsx:0356
src/pages/Devices.tsx:0357                    return () => {
src/pages/Devices.tsx:0358                        disposed = true;
src/pages/Devices.tsx:0359                        controller?.abort();
src/pages/Devices.tsx:0360                        clearInterval(interval);
src/pages/Devices.tsx:0361                    };
src/pages/Devices.tsx:0362                }, [liveModeEnabled]);
src/pages/Devices.tsx:0363
src/pages/Devices.tsx:0364                const openSettings = async (device: Device) => {
src/pages/Devices.tsx:0365                    setSelectedDevice(device.deviceId);
src/pages/Devices.tsx:0366                    setSettingsDeviceId(device.deviceId);
src/pages/Devices.tsx:0367                    setSettingsOpen(true);
src/pages/Devices.tsx:0368                    setConfigDraft(defaultConfig(device));
src/pages/Devices.tsx:0369                    await loadDeviceConfig(device.deviceId);
src/pages/Devices.tsx:0370                };
src/pages/Devices.tsx:0371
src/pages/Devices.tsx:0372                const closeSettings = () => {
src/pages/Devices.tsx:0373                    setSettingsOpen(false);
src/pages/Devices.tsx:0374                    setSettingsDeviceId(null);
src/pages/Devices.tsx:0375                    setConfigDraft(null);
src/pages/Devices.tsx:0376                    setCommandHistory([]);
src/pages/Devices.tsx:0377                };
src/pages/Devices.tsx:0378
src/pages/Devices.tsx:0379                const sendCommand = async (deviceId: string, command: string) => {
src/pages/Devices.tsx:0380                    setError(null);
src/pages/Devices.tsx:0381                    setActionMessage(null);
src/pages/Devices.tsx:0382                    setCommandBusyByDevice((current) => ({ ...current, [deviceId]: command }));
src/pages/Devices.tsx:0383                    try {
src/pages/Devices.tsx:0384                        if (PHYSICAL_CONFIRMATION_COMMANDS.has(command)) {
src/pages/Devices.tsx:0385                            const response = await requestDevicePhysicalConfirmation(deviceId, command);
src/pages/Devices.tsx:0386                            setActionMessage(
src/pages/Devices.tsx:0387                                response?.data?.status === 'pending_physical_confirmation'
src/pages/Devices.tsx:0388                                    ? `${commandLabel(command)} is waiting for board confirmation`
src/pages/Devices.tsx:0389                                    : `${commandLabel(command)} confirmation session created`
src/pages/Devices.tsx:0390                            );
src/pages/Devices.tsx:0391                        } else {
src/pages/Devices.tsx:0392                            const response = await sendDeviceCommand(deviceId, command);
src/pages/Devices.tsx:0393                            setActionMessage(response.message || `Command ${command} completed`);
```

frontend/src/pages/Devices.tsx:394-443

```
src/pages/Devices.tsx:0394                          }
src/pages/Devices.tsx:0395                          await fetchDevices();
src/pages/Devices.tsx:0396                          if (settingsOpen && settingsDeviceId === deviceId) {
src/pages/Devices.tsx:0397                              await loadDeviceConfig(deviceId);
src/pages/Devices.tsx:0398                          }
src/pages/Devices.tsx:0399                      } catch (e) {
src/pages/Devices.tsx:0400                          setError(getErrorMessage(e, 'FG-DEVICE-CMD-ERR'));
src/pages/Devices.tsx:0401                      } finally {
src/pages/Devices.tsx:0402                          setCommandBusyByDevice((current) => ({ ...current, [deviceId]: null }));
src/pages/Devices.tsx:0403                      }
src/pages/Devices.tsx:0404                  };
src/pages/Devices.tsx:0405
src/pages/Devices.tsx:0406                  const saveSettings = async () => {
src/pages/Devices.tsx:0407                      if (!settingsDeviceId || !configDraft) {
src/pages/Devices.tsx:0408                          return;
src/pages/Devices.tsx:0409                      }
src/pages/Devices.tsx:0410
src/pages/Devices.tsx:0411                      setConfigSaving(true);
src/pages/Devices.tsx:0412                      setError(null);
src/pages/Devices.tsx:0413                      try {
src/pages/Devices.tsx:0414                          const response = await saveDeviceConfig(settingsDeviceId, configDraft);
src/pages/Devices.tsx:0415                          setConfigDraft(response?.config || configDraft);
src/pages/Devices.tsx:0416                          setCommandHistory(response?.commandHistory || []);
src/pages/Devices.tsx:0417                          setActionMessage('Device configuration saved');
src/pages/Devices.tsx:0418                          await fetchDevices();
src/pages/Devices.tsx:0419                      } catch (e) {
src/pages/Devices.tsx:0420                          setError(getErrorMessage(e, 'FG-DEVICE-SAVE-ERR'));
src/pages/Devices.tsx:0421                      } finally {
src/pages/Devices.tsx:0422                          setConfigSaving(false);
src/pages/Devices.tsx:0423                      }
src/pages/Devices.tsx:0424                  };
src/pages/Devices.tsx:0425
src/pages/Devices.tsx:0426                  const selectedDeviceData = devices.find((device) => device.deviceId === selectedDevice) || null;
src/pages/Devices.tsx:0427                  const activeConfigDevice = devices.find((device) => device.deviceId === settingsDeviceId) || null;
src/pages/Devices.tsx:0428
src/pages/Devices.tsx:0429                  const onlineCount = devices.filter((device) => device.isOnline).length;
src/pages/Devices.tsx:0430                  const cloudCount = devices.filter((device) => device.isOnline && device.mode === 'online').length;
src/pages/Devices.tsx:0431                  const findingsOpen = devices.reduce((sum, device) => sum + (device.securitySummary?.asset?.findingsOpen || 0), 0);
src/pages/Devices.tsx:0432                  const recentActions = useMemo(
src/pages/Devices.tsx:0433                      () => devices.filter((device) => device.latestCommand || device.securitySummary?.latestCommand).length,
src/pages/Devices.tsx:0434                      [devices]
src/pages/Devices.tsx:0435                  );
src/pages/Devices.tsx:0436
src/pages/Devices.tsx:0437                  return (
src/pages/Devices.tsx:0438                      <div className="relative p-6 space-y-6 max-w-[1600px] mx-auto">
src/pages/Devices.tsx:0439                          <div className="flex justify-between items-center gap-4">
src/pages/Devices.tsx:0440                              <div>
src/pages/Devices.tsx:0441                                  <h2 className="text-2xl font-bold text-foreground tracking-tight">{t('devices.title')}</h2>
src/pages/Devices.tsx:0442                                  <p className="text-sm text-muted-foreground mt-1">设备、命令、配置与安全上下文统一查看。</p>
src/pages/Devices.tsx:0443                              </div>
```